

The Trader Book

An autonomous paper-trading system, explained for its founder

Contents

Chapter 1. What this system is, and what it is not

- [1. The idea in plain words](#)
- [2. Why it matters financially](#)
- [3. How this system does it](#)
- [4. What the evidence says so far](#)
- [5. Further reading](#)

Chapter 2. Markets, bars and costs

- [1. The idea in plain words](#)
- [2. Why it matters financially](#)
- [3. How this system does it](#)
- [4. What the evidence says so far](#)
- [5. Further reading](#)

Chapter 3. Features: turning prices into numbers

- [1. The idea in plain words](#)
- [2. Why it matters financially](#)
- [3. How this system does it](#)
- [4. What the evidence says so far](#)
- [5. Further reading](#)

Chapter 4. Labels: what "the model predicts" means

- [1. The idea in plain words](#)
- [2. Why it matters financially](#)
- [3. How this system does it](#)
- [4. What the evidence says so far](#)
- [5. Further reading](#)

Chapter 5. The forecaster

- [1. The idea in plain words](#)
- [2. Why it matters financially](#)
- [3. How this system does it](#)

[4. What the evidence says so far](#)

[5. Further reading](#)

Chapter 6. Honest validation

[1. The idea in plain words](#)

[2. Why it matters financially](#)

[3. How this system does it](#)

[4. What the evidence says so far](#)

[5. Further reading](#)

Chapter 7. The gate stack and meta-labeling

[1. The idea in plain words](#)

[2. Why it matters financially](#)

[3. How this system does it](#)

[4. What the evidence says so far](#)

[5. Further reading](#)

Chapter 8. Sizing, exits and risk

[1. The idea in plain words](#)

[2. Why it matters financially](#)

[3. How this system does it](#)

[4. What the evidence says so far](#)

[5. Further reading](#)

Chapter 9. The life of a trading day

[9.1 The idea](#)

[9.2 Why it matters financially](#)

[9.3 How this system does it](#)

[9.4 A worked example](#)

[9.5 What the evidence says](#)

[9.6 Further reading](#)

Chapter 10. From data to a deployed model

[10.1 The idea](#)

[10.2 Why it matters financially](#)

[10.3 How this system does it](#)

[10.4 A worked example: why 20 trades cannot certify anything](#)

[10.5 What the evidence says](#)

[10.6 Further reading](#)

Chapter 11. Measuring yourself honestly

[11.1 The idea](#)

[11.2 Why it matters financially](#)

[11.3 How this system does it](#)

[11.4 A worked example: 720 rows or 30?](#)

[11.5 What the evidence says](#)

[11.6 Further reading](#)

Chapter 12. The LLM as a bounded advisor

[12.1 The idea](#)

[12.2 Why it matters financially](#)

[12.3 How this system does it](#)

[12.4 A worked example: last night's bill](#)

[12.5 What the evidence says](#)

[12.6 Further reading](#)

Chapter 13. Running on an 8 GB Jetson

[13.1 The idea](#)

[13.2 Why it matters financially](#)

[13.3 How this system does it](#)

[13.4 A worked example: the memory ledger during a retrain](#)

[13.5 What the evidence says](#)

[13.6 Further reading](#)

Chapter 14. What the evidence says today, and what would change it

[14.1 The idea](#)

[14.2 Why it matters financially](#)

[14.3 How the result was reached](#)

[14.4 The levers](#)

[14.5 The Phase-3 retrain](#)

[14.6 How to read the next certificate](#)

[14.7 A worked example: the arithmetic of skill](#)

[14.8 The crypto decision](#)

[14.9 What would change the verdict](#)

[14.10 Further reading](#)

Chapter 15. A reading path to financial literacy for this system

[15.1 The idea](#)

[15.2 Why it matters financially](#)

[15.3 How this system's vocabulary maps to the glossary](#)

[15.4 A ten-item curriculum](#)

[15.5 The five traps, with examples from this repository](#)

[15.6 What the evidence says about learning this way](#)

[15.7 Further reading](#)

Chapter 1. What this system is, and what it is not

Part One of "the book": an explanatory guide to this trading system and the finance behind it. Written 2026-09-27 against the live tree on the Jetson. When this chapter and the code disagree, the code wins; where the older docs and the code disagree, the chapter says so.

1. The idea in plain words

This repository is a robot that tries to make small, repeated, disciplined bets on prices going up. It watches two groups of assets, which the code calls **books**:

- the **crypto book**: six coins traded against the US dollar (BTC, ETH, XRP, SOL, DOGE, LINK), open 24 hours a day, 7 days a week;
- the **stock book**: US stocks, traded only during regular market hours (9:30 to 16:00 New York time), chosen each hour from a larger panel of liquid names.

Every 30 seconds the robot wakes up, looks at the most recent completed hourly price bars, asks a forecasting model "how much is this asset likely to move over the next day or so?", and then pushes each candidate trade through a long list of checks before it ever sends an order. Most candidates are rejected. The ones that survive are bought in small sizes, protected by stop orders, and sold later by a fixed set of exit rules.

A few terms, defined once:

- **Paper trading**. Trading with pretend money at a real broker. The broker here is Alpaca. Orders are matched against real market quotes, but no real dollars move. Alpaca's own documentation says paper orders fill against the best available quote, a limit buy fills only when the limit is at or above the ask, about 10% of orders get random partial fills, and there is no market impact or queue position (recorded in `research/campaign_2026-09_jetson/research_engine.md`, source W12-S2). So paper fills are, if anything, kinder than real ones.
- **Long-only**. The system only ever buys something and later sells what it owns. It never **short-sells** (borrows shares and sells them, hoping to buy back cheaper). A short-side exit kernel exists in `policy_exits.py` (`side=-1`) but has no production caller.
- **Book**. A self-contained trading program with its own universe, hours, rules and model. The two books share one Alpaca account and one equity number.
- **Edge**. The average amount a trade makes after all costs. A system has an edge only if that average is reliably above zero. Everything in this book circles back to that one word.

What it is **not**:

- It is not a proven money machine. As of today it has no certified forecasting model in either book (section 4 explains why that is the honest result).
- It is not high-frequency trading. It reasons on **hourly bars** (one summary of price per hour) and holds positions for hours to about two days.
- It does not use leverage on purpose, options, or shorting. Position sizes are capped in dollars (section 3).
- It is not "run by an AI chatbot." Large language models (LLMs) appear only as a bounded advisor that can veto an entry or nudge its size (Chapter 7). The core forecast is a conventional statistical model (Chapter 5).

2. Why it matters financially

Why be so explicit about what the system is? Because most retail trading systems fail for one of three reasons, and each one has a direct counterpart here.

Failure 1: confusing market exposure with skill. Suppose the stock book holds seven positions of \$5,000 each (\$35,000 total) in high-beta names. **Beta** measures how much a stock tends to move when the whole market moves: a beta of 1.5 means that when the S&P 500 (tracked by the ETF **SPY**) falls 2%, the stock tends to fall about 3%. Then on a day when SPY drops 2%, this book loses roughly

$$\$35,000 \times 1.5 \times 2\% = \$1,050$$

without any forecast being wrong. In a rising year the same arithmetic produces gains that look like genius. The 2026-07 six-agent review concluded that about 85 to 95% of the invested variance in this system is exactly this kind of **factor variance** (moves explained by the market or by "crypto as a whole"), not stock-specific skill (docs/MAP.md §1). A system that does not measure this will believe it is smart in bull markets and unlucky in bear markets. That is why `beta_ledger.py` exists: it regresses daily equity on SPY and BTC and reports the **alpha** (the return left over after the market's share is removed) with an honest t-statistic.

Failure 2: ignoring costs. Every round trip (buy, then sell) in crypto costs about 0.60% of the position at this broker once fees and the bid/ask spread are included (Chapter 2 derives the number). A model that is right on average by 0.40% per trade loses money. A system that "trades often because the model is confident" can burn an account through costs alone.

Failure 3: fooling yourself in testing. If you try 40 random strategies on the same history, the best one will look good by pure luck. Chapter 6 shows the arithmetic: the luckiest of 40 coin-flip strategies with 60 trades each typically shows an annualized Sharpe ratio around 2, which is a number people quit their jobs over.

The design of this repository is a set of defenses against those three failures, in that order of subtlety. Paper trading is the fourth defense: nothing here should touch real money until the measurements say it has earned it.

3. How this system does it

The two machines and the one box that matters

Production runs on an **NVIDIA Jetson Orin Nano**, a small computer with 8 GB of memory shared between its processor and its graphics chip (GPU). The GPU trains models, one training process at a time (`gpu_lock.py` holds a file lock so two trainers never share the 8 GB). The trading bots run on the processor only: `CUDA_VISIBLE_DEVICES=' ', TORCH_NUM_THREADS=2, OMP_NUM_THREADS=2` (docs/MAP.md §4i). A development Mac is used for code and tests that need no heavy libraries. The owner's stated priority order is: **Jetson memory and performance, then financial soundness, then LLM utilization, then trading strategy** (CLAUDE.md). Notice that "trading strategy" is last. The system is built to be a trustworthy instrument first and a strategy second.

The process tree

- `run_pipeline.py` is the orchestrator. Its phases: **harvest** data (`scripts/harvest_crypto_data.py`, `scripts/harvest_stock_data.py`), **train** a model (`scripts/hypersearch_v2.py`), train the **meta-labeler** (`meta_label.py`), run the **promotion gate** (`backtest.py --gate`), then launch the bots and wait. Weekly it stops the bots, retrains, and restarts them (`run_pipeline._stop_bots` then `_restart_bots`).
- `run_bots.py` runs both trading loops as threads in one process, which saves about 0.5 to 0.8 GB of duplicated library memory on the Jetson.

- `base_loop.BaseTradingLoop` is the shared engine; `crypto_loop.CryptoLoop` and `stock_loop.StockLoop` fill in the book-specific parts (hours, universe, order style). The pattern is called **Template Method**: the base class owns the skeleton of each cycle, the subclasses override specific steps.

One cycle, in order

`base_loop._run_one_cycle` does roughly this every `LOOP_INTERVAL = 30` seconds (`base_loop.py`, `stock_loop.py`): check for a remote flatten request, check market hours, check the **circuit breaker** (a 5% daily account drawdown stops new buys, `CIRCUIT_BREAKER_PCT = 0.05`), manage exits on open positions **first**, hot-reload a newly promoted model if one exists, fetch predictions for every symbol, run the LLM advisor on its own 600-second timer (`LLM_INTERVAL_SEC = 600`), execute signal sells, then evaluate new buys through the gate funnel (Chapter 7) and size them (Chapter 8). Exits come before entries by design: protecting what you own matters more than adding to it.

The universe of each book

- Crypto: `stock_config.CRYPTO_SYMBOLS` lists the six coins above.
- Stocks: the harvest builds a panel of liquid US names (92 in the 2026-09-27 store) and applies an **as-of membership mask**: at each date, only the top 60 names by trailing 30-day dollar volume (`stock_config.AS_OF_TOP_K = 60`) and only names with at least \$5M daily dollar volume and a price of at least \$3 count as members (`scripts/harvest_stock_data._asof_tradability_mask`, `_asof_membership_mask`). Live, the stock loop ranks candidates each hour and considers entries only for the top 7 (`stock_loop.StockLoop.TOP_N = 7`), holding while a name stays in the top 15 (`HOLD_RANK = 15`).
- Stock entries are further restricted to two windows, 09:45 to 11:00 and 14:30 to 15:30 New York time (`strategy_config.STOCK_ENTRY_WINDOWS_ET`, `ENTRY_WINDOWS_ENABLED = True`), and positions are flattened near the close except for a small **overnight sleeve** of at most two positions (`OVERNIGHT_SLEEVE_MAX_POSITIONS = 2`).

Money limits that bind before anything clever happens

- Risk per trade: `RISK_PCT_PER_TRADE = 0.005`, i.e. 0.5% of equity lost if the stop is hit.
- Dollar caps per position: the base loop's `NOTIONAL_PER_SYMBOL = 1000` and `MAX_NOTIONAL_PER_SYMBOL = 3000` apply to crypto; the stock loop sets both to 5000.
- Per-book correlation-aware risk cap: `MAX_BOOK_RISK_PCT = 0.025` (2.5% of equity summed across a book's stops, adjusted for how correlated they are).

Worked example: with \$122,202 of equity (the paper account's cash after this morning's liquidation, per `research/campaign_2026-09_jetson/README.md` §6), 0.5% risk is \$611. If a stock's stop sits 1.2% below the entry, the "risk-based" size would be $\$611 / 1.2\% = \$50,917$. The \$5,000 cap cuts that to \$5,000, which risks only $\$5,000 \times 1.2\% = \60 , about 0.05% of equity. In practice the dollar caps bind first, so the book runs a fraction of its configured risk (`docs/MAP.md` §5-10 says roughly one third to one fifth). That is deliberate caution, and it also means the account cannot be hurt quickly while the system is unproven.

Every decision is written down

`trade_journal.log_decision` appends one JSON line per decision to `journals/YYYY-MM-DD.jsonl`: every buy, every sell, and every **skip** with the reason it was skipped. The measurement tools (`decision_report.py`, `beta_ledger.py`, `llm_eval.py`, `execution_report.py` and others) read these journals later. The principle is that every strategy change should be an **evidence-gated flag flip**: new behavior ships switched off (a "default-

OFF flag" in `strategy_config.py`) and is switched on only when a pre-registered measurement says so (`docs/FLAGS.md`, `research/campaign_2026-08/03_jetson_runbook.md`).

Fail-closed and fail-open

Two phrases you will meet in every chapter:

- **Fail-closed:** when information is missing, do nothing. A missing prediction, missing quote or missing model means **no new entry** (`docs/MAP.md` §6, invariant 3).
- **Fail-open:** when an optional advisor breaks, carry on as if it had said "neutral." The LLM gate and the meta-label gate are fail-open, so a provider outage can never block the whole book.

4. What the evidence says so far

Honesty first, because the rest of the book depends on it.

- **The 2026-07 review verdict still stands.** The deployed policy is "a gated, conditional-beta, long-only book": six correlated cryptos plus the top 7 of a set of high-beta stocks. Positive expected value (+EV) is unproven; the validation reviewer's honest probability that a full gate pass reflects real skill was about 40 to 55%, and a live Sharpe typically comes in 50 to 73% below its backtest (`docs/MAP.md` §1 and §9 item 1). The parts of the system that are reliably valuable today are **cost engineering** (cheaper order placement) and **loss avoidance** (the gates).
- **The device was stale until last night.** The bots had not run since 2026-05-07; the April models were LSTM-only with no manifest; the data stores ended in February on an old schema; and an audit found a one-day look-ahead leak in the sentiment feature that the April models had trained on (`research/campaign_2026-09_jetson/README.md` §1 and §2; Chapter 3 tells that story).
- **The clean rebuild found no selection skill.** After rebuilding both data stores from scratch (263,889 crypto rows; 1,536,356 stock rows over 92 names), a 40-trial crypto search produced 43 negative trials out of 44, and the best trial's holdout produced just 3 trades, too few to certify, so nothing was saved. A 25-trial stock search reached a holdout Sharpe of 0.13 with a Deflated Sharpe of 0.058 against a required 0.60, so nothing was saved either. The campaign's summary: "on clean, leak-free data the current LSTM + objective design shows no measurable selection skill in either book" (`README` §4). Chapter 6 explains, kindly, why that is a good outcome for the process even though it is a disappointing one for the strategy.
- **Operational state this morning** (`README` §6): on the founder's instruction the paper account was liquidated to cash (\$122,202), a stock-only systemd user service was installed and started, and a 70-trial stock retrain was launched with the "Phase-3 bundle" of training fixes switched on (`strategy_config.HYPERSEARCH_V3 = True`, `OBJECTIVE_V3 = True`, `TRAINING_REPAIRS_V1 = True`, `OBJECTIVE_LONG_ONLY = True`). Note for readers of the older docs: `docs/MODULES.md` §5 still says these are "all default OFF"; the code now has them ON, and the code wins. At the time of writing that retrain was a handful of trials in, still in its random-search phase, with no result yet. Until a model passes the gates, the stock bot runs but cannot buy: no model means no prediction, and no prediction means no entry (fail-closed).
- **Direction:** the founder has asked to stop spending effort on the crypto book and concentrate on stocks, supported by the crypto search result and by a slippage study showing real crypto crossing costs well above the model's assumption (Chapter 2).

The honest goal, then, is not "make money next week." It is: build an instrument that can tell the difference between skill and luck, that pays only for edge it can measure, and that runs unattended on an 8 GB box without hurting the account while it learns. Measured against that goal, last night was a success: the instrument worked

and correctly refused to certify a model that had no edge.

5. Further reading

- Marcos López de Prado, *Advances in Financial Machine Learning*, Wiley, 2018. The single most influential source for this repository's labeling, sample weighting, validation and meta-labeling.
- Larry Harris, *Trading and Exchanges: Market Microstructure for Practitioners*, Oxford University Press, 2003. How markets, orders, spreads and brokers actually work.
- Ernest P. Chan, *Algorithmic Trading: Winning Strategies and Their Rationale*, Wiley, 2013. A practical, honest introduction to building and testing retail-scale trading systems.
- David H. Bailey, Jonathan M. Borwein, Marcos López de Prado and Qiji Jim Zhu, "Pseudo-Mathematics and Financial Charlatanism: The Effects of Backtest Overfitting on Out-of-Sample Performance," *Notices of the American Mathematical Society* 61(5), 2014. Why impressive backtests are cheap.
- Campbell R. Harvey, Yan Liu and Heqing Zhu, "...and the Cross-Section of Expected Returns," *Review of Financial Studies* 29(1), 2016. How many published "discoveries" are false positives.

Chapter 2. Markets, bars and costs

Why a crypto trade must be predicted to move more than 1.2% before this system will even consider it, and what last night's fill study says about whether even that is enough.

1. The idea in plain words

A market is a place where buyers and sellers leave prices. At any moment there is a best price someone will pay (the **bid**) and a best price someone will accept (the **ask**). The ask is always a little above the bid. The gap is the **spread**, and the point halfway between is the **midpoint** ("mid"). If Bitcoin shows a bid of \$59,990 and an ask of \$60,010, the spread is \$20 and the mid is \$60,000. We usually express the spread as a percent of the mid: $\$20 / \$60,000 = 0.033\%$, or 3.3 **basis points** (a basis point, "bp" or "bps", is one hundredth of one percent, 0.01%).

If you buy at the ask and immediately sell at the bid, you lose the whole spread without the price having moved at all. That is the first cost of trading. The second is **fees**, which the broker or exchange charges per order. The third is **slippage**: the difference between the price you expected (say, the mid when you decided) and the price you actually got.

A **bar** is a summary of trading over a fixed period. An **hourly bar** records four prices, the **open** (first trade of the hour), **high**, **low** and **close** (last trade), plus the **volume** traded. Together these are called **OHLCV**. This system thinks only in hourly bars. It never looks at individual trades or the order book.

Two kinds of order matter here:

- a **taker** order crosses the spread and fills immediately against someone else's resting order (a market order, or a limit order priced at or through the far side);
- a **maker** order rests on the book at your chosen price and waits for someone else to come to you. Makers usually pay lower fees, but they may never fill, and when they do fill it is often because the price is about to move against them (this is called **adverse selection**).

2. Why it matters financially

Costs are paid on every trade, win or lose, while edge is only an average. That asymmetry decides almost everything about which strategies can work.

The crypto arithmetic. At Alpaca's tier-1 crypto schedule a taker pays 25 bps per side (`fees.CRYPTO_TAKER_BPS = 25.0`). A round trip (buy then sell) therefore costs 50 bps in fees. Add a spread allowance of 0.10% (`fees.FLAT_SPREAD_PCT['crypto'] = 0.10`) and the model's cost of one round trip is

$0.25\% + 0.25\% + 0.10\% = \mathbf{0.60\%}$ of the position.

So a trade whose true expected move is +0.50% loses 0.10% on average. A strategy with a genuine, repeatable +0.50% predictive edge per trade, which would be a very good forecaster, is a losing strategy in crypto at these fees. This is the "0.6% edge" you will see quoted all over the repo: it is the **break-even** move for a crypto trade.

The stock arithmetic. US stocks at Alpaca carry no commission, only tiny sell-side regulatory fees (`fees.STOCK_REGULATORY_BPS = 0.3` per round trip) plus a slippage allowance of 3 bps per side (`STOCK_SLIPPAGE_BPS_PER_SIDE = 3.0`) and a default spread of 0.05% (`FLAT_SPREAD_PCT['stock'] = 0.05`):

$0.003\% + 0.03\% + 0.03\% + 0.05\% = \mathbf{0.113\%}$ per round trip.

Stocks are about five times cheaper to trade than crypto here. That single fact explains a lot of last night's results.

Why demand twice the cost. A forecast is noisy. If you admit every trade whose predicted move is just above break-even, half of them will in reality fall below it, and the average admitted trade will barely pay its costs. The system therefore demands that the predicted move be at least **twice** the round-trip cost before entering (`fees.MIN_EDGE_MULTIPLE = 2.0`). For crypto that is a 1.20% **admission floor**; for stocks, 0.226%. A 2026 hourly-Bitcoin study (Bysik and Ślepaczuk, arXiv 2606.00060) independently found that requiring the predicted move to exceed a fixed 2.0 times cost "sharply reduces turnover and restores profitability," which the campaign notes as outside support for this exact multiple (`research/campaign_2026-09_jetson/research_signal.md`, SCOUT-4 T1).

What goes wrong without it. Before the cost model was rebuilt, the training objective assumed a 5 bp round trip (`scripts/hypersearch_v2.py` comment above `TXN_COST_PCT`), about a tenth of crypto reality. It selected models whose "edge" could not survive live fees. The `fees.py` module docstring records the older live gate compared predictions against the spread alone, "admitting structurally negative-expectancy crypto trades."

3. How this system does it

Bars: closed, aligned, point-in-time

- `market_data.drop_forming_bar` removes the hour that is still in progress. The model only ever sees **closed** bars, because a half-formed bar's "close" is not a close at all.
- Crypto bars come from Alpaca with a yfinance fallback; stock bars come from Alpaca, with yfinance used only as a fallback because its hourly stock bars are stamped on the half hour (:30) while Alpaca's are on the hour (:00) (`docs/MAP.md` §4a).
- Crypto has $24 \times 365 = 8,760$ hourly bars per year. The trainer annualizes stocks with 1,638 bars per year (`hypersearch_v2.BARS_PER_YEAR`), which is 6.5 regular-session hours on 252 days. Last night's campaign measured that the rebuilt stock store actually holds extended-hours bars, about 3,827 per year (`bars_calendar.py`, flag `BARS_PER_YEAR_MEASURED`, default OFF). This matters because a Sharpe ratio is annualized with the square root of bars per year; using the wrong count mis-scales every stock Sharpe by about $\sqrt{3827/1638} = 1.5$.

One cost function for everyone

The whole system asks a single module what a trade costs:

- `fees.round_trip_cost_pct(asset_type, spread_pct, maker, live)` returns the raw cost **charged to P&L**: fee constant plus the full spread, charged once per round trip. For crypto the entry fee is taker (25 bps) unless the maker flag is set or, in the live gate only, blended by the recent share of entries that actually filled as maker (`fees.crypto_entry_fee_bps(live=True)` reads the journals; with fewer than 30 recent entries it assumes all taker). The exit side is always priced taker.
- `fees.required_edge_pct(...)` returns the **admission floor**: that cost times `MIN_EDGE_MULTIPLE`, resolved at call time so every caller moves together.

The docstring of `fees.py` calls this a "cost-multiple ladder": 1.0x cost is charged to P&L in backtests and meta-labels, 2.0x cost is the entry floor, and an extra 1.5x headroom is declared for a future passive-order tactic. Two deliberate copies exist and are pinned by tests rather than imported: the trainer's `TXN_COST_PCT = {'crypto': 0.60, 'stock': 0.11}` and `backtest.SPREAD_PCT`.

The live cost gate

In the live loop, `order_utils.should_trade(pred_return, spread_pct, asset_type=...)` computes the floor from the **live quoted spread** (`required_edge_pct(..., live=True)`, the only production `live=True` caller) and rejects the candidate if the predicted move does not exceed it. The rejection is journaled as a `cost_floor skip` with the spread at that moment, so the decision report can later price what was given up (`base_loop._execute_buys`).

Quotes themselves are sanity-checked in `order_utils.get_quote`: a quote older than 180 seconds, or non-positive, or NaN is rejected, and a missing quote means no entry (fail-closed).

Estimating the spread from bars: EDGE

Historical quotes are not always available, but historical OHLC bars are. The **EDGE** estimator (Ardia, Guidotti and Kroencke, *Journal of Financial Economics*, 2024) infers the effective spread from the pattern of open, high, low and close prices: roughly, bid/ask bounce leaves a fingerprint in how closes relate to the surrounding highs and lows. `liquidity.edge_spread_series(ohlc_df, window=35)` computes a trailing, per-bar estimate with the bidask package and clips it to `[SPREAD_FLOOR_PCT = 0.02, SPREAD_CAP_PCT = 1.50]` percent. The stock harvest stamps it into the column `Eff_Spread_Pct` so that backtests charge each historical trade the spread of its own time and name rather than one flat number (the two-tier cost model: the flat spread sets the admission floor, the per-bar spread is what gets charged, docs/MAP.md §4c).

An honest limit, from the 2026-08 literature pass (`research/campaign_2026-08/02_research.md` B05.1): EDGE's statistical noise on hourly bars creates a spurious floor of roughly 0.10 to 0.20% for mega-cap stocks, five to ten times their true 1 to 5 bp spreads. That errs on the conservative side (it overstates cost), but it narrows the tradable universe. The fix, estimating from one-minute bars, is built dark behind `TRADER_STOCK_MINUTE_EDGE` and awaits an owner ruling. Crypto is not stamped at all today; it uses the flat 0.10%.

Order placement

Crypto entries use a **maker ladder** when `MAKER_ENTRIES_ENABLED = True`: the bot first joins the bid and waits up to `MAKER_STAGE_TIMEOUT = 25` seconds per rung (two rungs), then falls back to a taker order (`order_utils.place_maker_buy`). Stock entries are **bracket** limit orders that carry a stop-loss leg and a take-profit leg with them. `execution_report.py` measures **implementation shortfall**, the difference between the price at the moment of decision and the realized fill (Perold, 1988).

A worked example

It is 14:00 UTC. The model predicts that ETH will rise 0.90% over its horizon. The live quote shows a spread of 0.024% (2.4 bps, typical of ETH on Alpaca, see section 4).

1. Round-trip cost, all taker: $0.25 + 0.25 + 0.024 = 0.524\%$.
2. Admission floor: $2 \times 0.524 = 1.048\%$.
3. The prediction of 0.90% is below 1.048%, so `should_trade` returns False and the loop journals a `cost_floor skip`.

The same forecast on a US stock with a 0.03% quoted spread faces a cost of $0.063 + 0.03 = 0.093\%$ and a floor of 0.186%, which 0.90% clears easily. The model is equally "confident" in both cases; only the market's price of admission differs.

4. What the evidence says so far

The admission floor and the model's own scale. Last night's diagnosis of the crypto search (DECOMP-1, `research/campaign_2026-09_jetson/README.md` §4) found that 44 of 48 fold Sharpes were negative and that **every threshold the search tried was below the live 1.2% floor**. In other words the search was selecting models whose trades would be charged more than they could earn. SCOUT-4 (`research_signal.md` T1) confirmed the mechanism in code: the trainer scores "pred > threshold" with a cost of 0.60%, while the live book and the backtest require "pred >= max(threshold, 1.20%)." The legacy threshold range [0.05, 1.0] puts 100% of crypto draws below the floor by construction. The Phase-3 bundle now switched on (`OBJECTIVE_V3 = True`) re-anchors the searched range to 0.8x to 2.5x the floor (`objective_utils.v3_trade_threshold_range`); for stocks that is 0.18% to 0.57%, and the stock retrain running this morning is indeed drawing thresholds in that band.

The real price of crossing in crypto. The ENGINE department reconstructed **686 real paper crypto fills** from February to April 2026 using the broker's own records and historical quotes on two Alpaca venues (`research_engine.md` §X2). Findings:

- Market-order fills sit at the Alpaca us venue's touch (median 0.4 bps better than its ask), and about 12.7 bps beyond the Kraken-routed us -1 ask. Paper crypto is priced off the us book.
- The us book's spreads are much wider than us -1 for some coins, and the gap is structural, not a weekend effect: XRP about 39 bps versus under 1 bp; DOGE about 33 versus under 1; LINK about 20 versus 4; BTC and ETH only 2 to 4 bps.
- Measured round-trip crossing cost (twice the median taker slippage): **LINK 30 bps, XRP 26 bps, DOGE 34 bps**, against the model's flat 10 bps spread allowance.
- The fee records imply a median of about 22 bps on market orders and 12 to 13.5 bps on limit fills, not the published 25/15. The report marks this "unexplained, unverified" and does not use it.
- Maker fills: zero. The maker ladder was added after the last time the bots ran, so there is no evidence yet on whether maker entries save money after adverse selection.

The honest summary: for BTC and ETH the cost model is about right; for the smaller coins it understates the true spread cost by a factor of two to three. Combined with the search result, this is why the founder directed the effort toward the stock book, where the cost of admission is about a fifth as large. These are paper fills, so none of this licenses a change to live cost settings; the study's pre-registered rule says to rerun it on at least 30 live fills if the account ever goes live.

5. Further reading

- Larry Harris, *Trading and Exchanges: Market Microstructure for Practitioners*, Oxford University Press, 2003. Chapters on bid/ask spreads, order types and transaction costs.
- David Ardia, Emanuele Guidotti and Tim A. Kroencke, "Efficient Estimation of Bid-Ask Spreads from Open, High, Low, and Close Prices," *Journal of Financial Economics* 161, 103916, 2024. The EDGE estimator used in `liquidity.py`.
- Richard Roll, "A Simple Implicit Measure of the Effective Bid-Ask Spread in an Efficient Market," *Journal of Finance* 39(4), 1984. The original idea that bid/ask bounce is visible in prices.
- André F. Perold, "The Implementation Shortfall: Paper versus Reality," *Journal of Portfolio Management* 14(3), 1988. Why the price you decided at is not the price you get.
- Andrea Frazzini, Ronen Israel and Tobias J. Moskowitz, "Trading Costs," SSRN working paper 3229719,
 1. What trading really costs a large institution, measured from \$1.7 trillion of executions.

Chapter 3. Features: turning prices into numbers

What the model actually sees, why most of it is ratios and ranks rather than prices, and the story of a one-day leak that sat inside every model trained before last night.

1. The idea in plain words

A forecasting model cannot look at a chart. It needs a table of numbers. Each row of the table is one moment in time for one asset (for example "SOL at the close of the 14:00 UTC bar"), and each column is a **feature**: a number computed from information available at that moment. The model learns a mapping from a row of features to the future return that followed.

Three rules govern good features:

1. **Stationarity.** A series is **stationary** when its statistical behavior (average, spread) does not drift over time. The raw price of Bitcoin is not stationary: it was \$30,000 in one year and \$100,000 in another, so a model that learns "high price means X" learns nothing that transfers. A return ("up 1.2% over 4 hours") or a ratio ("2% above its 20-hour average") behaves similarly in any year. This system trains only on the stationary kind.
2. **Point-in-time (PIT) discipline.** Every feature must be computable using only data that existed at that moment. If a feature quietly uses tomorrow's information, the backtest will look brilliant and the live system will not. That mistake is called **look-ahead bias** or **leakage**.
3. **Train/serve parity.** The number the model sees in live trading must be computed exactly the way it was computed during training. Two slightly different formulas for "RSI" in two places is enough to make a good model behave badly.

2. Why it matters financially

Leakage produces fake edge. Here is the arithmetic of how little leakage it takes. Suppose a sentiment score for day D is accidentally built from news published up to the end of day D, and the model is predicting returns during day D. Even if the score explains only a sliver of the day's move, say one twentieth of the variance of a 24-hour return, that is a correlation of about $\sqrt{0.05} = 0.22$ between feature and target. A real, honest hourly forecasting signal typically has a correlation (**information coefficient**, IC) of 0.01 to 0.05 with forward returns. So one leaked column can dwarf everything honest in the table, and the model will lean on it. In live trading the leaked information does not exist yet, and the edge vanishes on the first day.

Survivorship bias produces fake edge too. If you train only on stocks that are in today's index, you have silently selected companies that survived and grew. A model trained on survivors learns that "beaten-down stocks bounce," because the ones that did not bounce were delisted and are missing from your data. The fix is an **as-of universe**: at each historical date, use only the names that would have qualified on that date.

Non-stationary features overfit. A model given raw prices can memorize "BTC around \$40,000 in 2023 tended to go up," which is a fact about one period, not a rule.

3. How this system does it

One function, two callers

All price-based features come from `indicators.compute_features(df, btc_close)` (the crypto and base block) and `indicators.compute_stock_features(df, spy_close, symbol)` (base plus session and daily features for stocks). The harvest scripts call them to build the training store, and `predict_now.get_live_prediction` calls the **same functions** on live bars. Warm-up gaps (rows where a long window has not filled yet) are filled with the same neutral values on both sides by `indicators.fill_warmup_features` (0.0 for most, 0.5 for range-position features). That is what makes parity structural rather than a matter of discipline (`docs/MODULES.md` §3). Changing any feature's values is **model-facing**: it requires a fresh harvest and a fresh training run.

The stationary preset

The harvest computes every indicator; a **preset** chooses which columns the model trains on. Production always trains with `--preset stationary` (hardcoded in `run_pipeline.py`, so the GUI's preset picker only affects manual runs). The list lives in `indicator_config.STATIONARY_FEATURES`. The trainer intersects it with the columns present in each book's store; the training logs show 30 features for crypto and 65 for stocks (`research_signal.md` SCOUT-2 C1). The families:

Returns and volatility. `Return_4h` and `Return_12h` are percent changes over 4 and 12 bars; `Volatility_12h` is the standard deviation of the last 12 hourly returns. `ROC` is a 12-bar rate of change, which the 2026-07 review found to be bit-for-bit identical to `Return_12h` (a duplicate the `stationary_lean` preset removes).

Price ratios. `Price_SMA20_Ratio` is the close divided by its 20-bar simple moving average (1.02 means 2% above the average). `Volume_Ratio` is volume over its 20-bar average. Bollinger bands are a 20-bar average plus and minus 2 standard deviations; `BBP_20_2.0` says where the close sits inside the band (0 at the lower band, 1 at the upper), and `BBB_20_2.0` is the band's width.

Oscillators. Bounded indicators from classic technical analysis:

- `RSI` (Relative Strength Index, Wilder, 14 bars): $100 - 100 / (1 + RS)$, where `RS` is the average gain divided by the average loss. Worked example: if the smoothed average hourly gain is 0.30% and the average loss is 0.20%, `RS` = 1.5 and `RSI` = $100 - 100/2.5 = 60$. Readings above 70 are conventionally "overbought," below 30 "oversold."
- `STOCHk_14_3_3` and `STOCHd_14_3_3` (stochastic oscillator): where the close sits within the last 14 bars' high-low range, smoothed.
- `MACD` (12, 26, 9): the difference between a 12-bar and a 26-bar exponential moving average (`MACD_12_26_9`), its 9-bar signal line (`MACDs`), and their difference, the histogram (`MACDh`). Note that `MACDs` = `MACD` - `MACDh` exactly, so three columns carry two numbers of information.

Time and calendar. Hours and weekdays are cyclical, so they are encoded as a point on a circle: `Hour_sin` = $\sin(2 \pi h / 24)$, `Hour_cos` = $\cos(2 \pi h / 24)$. Worked example: 14:00 gives `sin` = -0.500 and `cos` = -0.866. The benefit is that 23:00 and 00:00 end up close together, as they are in reality, instead of 23 units apart. `Day_sin/cos`, `Month_sin/cos` and `Turn_of_Month` follow the same idea. (The kill list records that month-of-year seasonality has decayed since 2015 and that a training span holds at most about five yearly cycles, which is why the lean preset drops it.)

Hurst exponent. `Hurst` is a rolling 100-bar estimate of whether a series trends (above 0.5) or mean-reverts (below 0.5). A code comment in `indicators.compute_features` is candid: the historical call feeds price *levels*,

for which a pure random walk reads about 0.8 regardless of regime, so the feature "carried no regime information." The correct version (on returns) is behind `indicator_config.HURST_ON_RETURNS = False` and would require a retrain.

Crypto positioning. From perpetual-futures markets (which the system does not trade, but reads): `Funding_Rate_Ann` (the 8-hour funding rate annualized, $\text{rate} \times 3 \times 365$), `Funding_Z` (its z-score against the trailing 90 prints), `Funding_Chg_24h`, open-interest change and z-score (`OI_Chg_24h`, `OI_Z`), a top-trader long/short z (`TT_LS_Z`) and taker flow imbalance (`Taker_Imb_24h`). A **z-score** is (value minus its recent average) divided by its recent standard deviation, so "+2" means "two standard deviations above normal." Very positive funding means crowded long positioning, which the cited research links to crash risk at this system's horizon.

Stock session and daily features. Price relative to VWAP, the overnight gap, ATR as a percent of price, relative strength against SPY, `ROD_Ret` (return since the prior session's close), `Same_Hour_Mean_40d` (the trailing 40-session average return of this same clock hour, shifted so the current bar is excluded, after Heston, Korajczyk and Sadka 2010), residual reversal (`RR_5`, `RR_21`), distances from 10- to 200-day moving averages, overnight momentum, range position, and FINRA short volume (`SVR_21`, the 21-day share of volume that was short sales, and its z-score `SVR_Z`).

Cross-sectional panel ranks (stocks). The stock book chooses the top 7 of many names each hour, so what matters is how a stock compares with the others *right now*. `panel_ranks.add_panel_ranks` ranks each base feature across all member stocks at the same timestamp and maps the rank to [-1, +1] with $2 \times (\text{rank} - 1) / (n - 1) - 1$ (`panel_ranks._signed_rank`). Worked example: a stock ranked 45th of 60 by 4-hour return gets $2 \times 44 / 59 - 1 = +0.49$, meaning "better than about three quarters of the panel." Context columns add the cross-sectional dispersion (`CS_Dispersion`) and breadth (`CS_Breadth`, the centered share of names above their 20-bar average). The raw dollar volume is dropped after ranking so only its rank is a feature. Ranking follows Gu, Kelly and Xiu (2020), who found rank-transformed characteristics work well for machine-learning return prediction.

Sentiment. `Daily_Sentiment`: for crypto, the Crypto Fear and Greed index mapped from 0 to 100 onto -1 to +1 (`sentiment_history._fng_value_to_score, (value - 50) / 50`); for stocks, a daily per-symbol news score from Finnhub headlines.

Point-in-time machinery

- **As-of universe (stocks):** at each date only the top 60 names by trailing 30-day dollar volume, with a \$5M dollar-volume and \$3 price floor, are members (`scripts/harvest_stock_data._asof_tradability_mask, _asof_membership_mask, panel_ranks.dv30`). `docs/MAP.md §6` calls these masks "the only place survivorship is prevented."
- **Publication lags:** sentiment is lagged to its publication date; short interest to its FINRA publication date; closed bars only (`market_data.drop_forming_bar`).

The sentiment leak story

This is the most instructive bug in the repository, because it was subtle, it was real, and it was found by an audit rather than by bad live results.

The crypto Fear and Greed value is published by `alternative.me` once a day at 00:00 UTC. The cache table `fng_daily` stored each value under a calendar date. Before a fix, rows were dated in the Jetson's **local time zone**, `America/Chicago`. 00:00 UTC is 19:00 or 18:00 the previous evening in Chicago, so a value published at the start of UTC day D+1 was filed under date D. Every hourly training bar on day D therefore saw a sentiment value that would not exist until the end of that day: up to 24 hours of look-ahead. The audit verified this on 1,875 of 1,875 cached days against the API (comment above `sentiment_history._migrate_fng_date_basis`). Worse, a later

fix to the insert code only governed new rows; cached rows were never rewritten, so every re-harvest re-leaked.

The stock side had a smaller cousin of the same bug: the news cache is Chicago-dated, so the aggregate for calendar day X includes articles up to about 05:00 to 06:00 UTC on X+1. The rule "use yesterday's score" was safe for regular-hours bars but not for the extended-hours bar at 00:00 UTC.

The repairs (research/campaign_2026-09_jetson/README.md §2 and §3):

- `sentiment_history._migrate_fng_date_basis` moves every legacy row into a side table (delete nothing), refills the table with UTC-publication-dated values in one transaction, and sets a marker so it runs once. It refuses to run if the refill covers less than 90% of the legacy rows, so a truncated API response can never replace real data with zeros.
- `sentiment_history.stock_sentiment_lookup_dates` keys each stock bar to `((t - 6h).date() - 1 day)` in UTC, which sends every bar before 06:00 UTC one more day back so the looked-up bucket is strictly in the past.

The April 2026 models had trained on the leaked column, which is one reason last night's clean rebuild was mandatory rather than optional.

4. What the evidence says so far

- **Leaks and data faults found and fixed last night** (research/campaign_2026-09_jetson/README.md): the sentiment look-ahead above; the stock harvest silently losing its newest three months (the consolidated-tape end date was not clamped); open-interest zero prints now masked; triple-barrier labels now stamped after row filters (Chapter 4). A C-language speed-up of the indicator kernels was found to overflow memory on short frames and was archived, now opt-in via `TRADER_INDICATORS_C=1`; the Numba version is the default and numerically identical.
- **The rebuilt stores:** crypto 263,889 rows by 71 columns; stocks 1,536,356 rows by 110 columns over 92 names, all from Alpaca bars, with `Eff_Spread_Pct`, the `CS_*` panel ranks and labels re-derived.
- **Features are not edge.** On those clean stores, the model built from these features showed no measurable selection skill in either book (Chapter 6). That does not prove the features are useless; it proves that this model, objective and cost structure did not turn them into profitable trades. `indicator_leadlag.py`, the tool that measures each feature's predictive correlation with forward returns at 1 to 48 hours with false-discovery control, has not yet been run to a verdict on the new stores.
- **The kill list constrains new ideas.** `research/KILL_LIST.md` rules out, among others, any additional technical oscillators ("A-minus grade negative evidence post-cost"), post-earnings drift, 52-week-high anchoring, and on-chain flow features. A new feature idea must be checked there first.
- **Known skews awaiting the owner:** `oi_archive.py` open-interest units differ between the training archive (Binance, dollar notional) and the live source (OKX, coin units); the funding z-score live baseline spans only about 2.8 days unless `TRADER_FUNDING_Z_TIME_THINNING` is enabled (`docs/MAP.md` §9, `docs/MODULES.md` §3). Both are crypto-side and now on hold.

5. Further reading

- John J. Murphy, *Technical Analysis of the Financial Markets*, New York Institute of Finance, 1999. Clear definitions of RSI, MACD, Bollinger bands and stochastics.
- J. Welles Wilder Jr., *New Concepts in Technical Trading Systems*, Trend Research, 1978. The original RSI and ATR.
- Shihao Gu, Bryan Kelly and Dacheng Xiu, "Empirical Asset Pricing via Machine Learning," *Review of*

Financial Studies 33(5), 2020. Rank-transformed features and what machine learning can and cannot extract from them.

- Steven L. Heston, Robert A. Korajczyk and Ronnie Sadka, "Intraday Patterns in the Cross-section of Stock Returns," *Journal of Finance* 65(4), 2010. The same-hour periodicity feature.
- Marcos López de Prado, *Advances in Financial Machine Learning*, Wiley, 2018, chapter 5 (stationarity and memory in features).
- Cheol-Ho Park and Scott H. Irwin, "What Do We Know About the Profitability of Technical Analysis?" *Journal of Economic Surveys* 21(4), 2007. A sober survey of the evidence.

Chapter 4. Labels: what "the model predicts" means

A forecast is only as meaningful as the question it answers. This chapter is about choosing the question, and about why the question used in training must be the same one the backtest scores.

1. The idea in plain words

In supervised machine learning, every training row has an **input** (the features of Chapter 3) and an **answer**, called the **label** or **target**. The model learns to produce answers like the labels from inputs like the features. Whatever the label measures, that is what "the model predicts."

This system offers the trainer two kinds of label, and the training search picks one per trial (`target_kind` in `scripts/hypersearch_v2.py`):

1. **The fixed-horizon forward return** ("raw"). For a horizon of `fb` bars, $\text{Target_Return_fb} = (\text{close}[t + \text{fb}] - \text{close}[t]) / \text{close}[t] \times 100$, the percent change from this bar's close to the close `fb` hours later (`scripts/harvest_crypto_data.py`; the stock harvest is the same). The searched horizons are 12, 18, 24, 32 and 48 bars (`adaptive_config.DEFAULT_SEARCH_SPACE ['forward_bars']`). This label answers: "if I bought now and held exactly `fb` hours, what would I make?"
2. **The triple-barrier return** ("tb"). Imagine three barriers around an entry: an upper barrier (take profit), a lower barrier (stop loss), and a vertical barrier (a time limit). The trade ends at whichever barrier price touches first. The label `TB_Ret_fb` is the gross percent return realized that way. This label answers: "if I bought now and let my actual exit rules run, what would I make?"

The second question is the one a trader cares about, because nobody in this system holds for exactly 24 hours. The bot exits on stops, trailing stops, take-profits, signal flips and, for stocks, the end of the day.

2. Why it matters financially

A worked example shows how far apart the two questions can be.

Bitcoin closes an hour at **\$60,000**. Its **ATR** (Average True Range, the average size of an hourly bar's full range including gaps, 14 bars) is **\$500**. The crypto exit policy in `strategy_config.CRYPTO_POLICY` says: stop at 2.5 ATR below entry, take profit at 2 times the stop distance, trailing stop at 2.0 ATR that arms once price is 1.5% above entry. So:

- stop distance = $2.5 \times 500 / 60,000 = 2.083\%$, stop price **\$58,750**;
- take-profit distance = $2 \times 2.083\% = 4.167\%$, take-profit price **\$62,500**;
- trailing distance = $2.0 \times 500 / 60,000 = 1.667\%$, arming level $60,000 \times 1.015 = \mathbf{\$60,900}$.

Now the price path. In hour 3 the high reaches \$61,400, which arms the trail. In hour 4 the bar opens at \$60,900 and trades down to \$60,200. The trailing stop sits at $61,400 \times (1 - 0.01667) = \mathbf{\$60,377}$, so the position exits there. Over the next day Bitcoin drifts down and closes hour 24 at **\$59,400**.

- The raw 24-bar label says: $(59,400 - 60,000) / 60,000 = \mathbf{-1.00\%}$. "Bad entry."

- The triple-barrier label says: $(60,377 - 60,000) / 60,000 = +0.63\%$ gross. "Good entry" (before the 0.60% round-trip cost, which is charged downstream, so +0.03% net).

A model trained on the first label is taught to avoid this entry; a model trained on the second is taught to take it. Only one of them describes what the bot would actually have experienced. Train on the wrong question and even a perfect forecaster of it can lose money, because it optimizes a game you are not playing.

The same logic forces a second requirement: **the labels, the backtest and the live exits must all use the same exit rules**. If the backtest scored trades with different stops than the labels assumed, the backtest would be grading the model on a different game from the one it studied, and a promotion decision based on it would be noise.

3. How this system does it

One exit kernel, several consumers

`policy_exits.exit_walk` is the single implementation of the exit stack. For an entry at **every** bar's close it walks forward bar by bar and records the exit bar, price and reason. It is compiled with Numba for speed and has an identical pure-Python fallback. Its consumers (`policy_exits.py` module docstring, `docs/MAP.md §4f`):

Consumer	Parameters	Question answered
harvest labels, <code>policy_exits.compute_tb_labels</code>	vertical barrier = fb bars, no signal exit	"what does the exit stack realize from here within fb hours?"
promotion backtest, <code>backtest.simulate_ticker</code>	no time limit, signal exit on	"what would this model's actual policy have made?"
meta-labeler, <code>meta_label._gen_meta_rows</code>	same as the backtest	"did acting on this signal make money after costs?" (Chapter 7)
gate attribution, <code>decision_report.replay_entry</code>	24-bar limit, no signal exit	"what did a skipped trade give up?"

Because they call one kernel with the same policy numbers from `strategy_config.py`, labels and backtest agree by construction. The labels are **gross** (before costs); each consumer charges costs itself using `fees.py` (Chapter 2).

The rules, precisely

The kernel's docstring lists "normative" rules that any reimplementations must match. The important ones, in plain words:

1. **Entry is at the bar's close**, and the walk starts on the next bar. The entry bar's own high and low are never examined (you cannot be stopped out by prices from before you bought).
2. **Each bar is checked in a fixed order**: stop first (the hard stop, or the trailing stop if it is armed and higher), then take-profit, then update the high-water mark and possibly arm the trail, then the signal-flip exit, then the end-of-day flatten. Checking the stop before the take-profit when both are touched in the same bar is the **conservative** choice: with only hourly data you cannot know which happened first, so assume the bad one.
3. **Gaps are honored**. If a bar opens below the stop, the fill is at the (worse) open, not at the stop price: `exit`

= min(open, stop).

4. **The high-water mark (HWM)** is the highest price since entry. It updates only after that bar's stop and take-profit checks, so a bar that arms or raises the trail can never also fire it.
5. **The vertical barrier** loses every same-bar tie: if a stop, signal or end-of-day exit lands on the last allowed bar, that reason wins.

The stop distances come from the policy dictionaries, clipped to sane bounds. For crypto (CRYPTO_POLICY): stop 2.5 ATR, trail 2.0 ATR, floor 1.5%, ceiling 15%, take-profit 2.0 times the stop distance capped at 30%, trail arming at +1.5%. For stocks (STOCK_POLICY): stop 2.0 ATR, trail 2.0 ATR, floor 1.0%, ceiling 10%, take-profit 2.0 times capped at 15%, trail arming at +1.0%. If ATR is missing the kernel falls back to fixed percentages (6% and 5% stop and trail for crypto).

Stocks: the end of day is a barrier too

compute_tb_labels treats each stock session's last bar as a barrier (an approximation of the live flatten near 15:50 New York time). A consequence documented in its docstring: because the day always ends within one session, every horizon longer than a session produces **identical** stock labels. For stocks, "predict 32 hours ahead" and "predict 48 hours ahead" are the same question under the triple-barrier label.

Live is a mirror, not a caller

The live loops do **not** import the kernel (a test, tests/test_ia4_flagged.py, pins this), because they must act on one 30-second quote at a time rather than on a completed history. They re-implement the same rules in base_loop._desired_stop_for and base_loop._manage_stops (plus copies in stock_loop), reading the same numbers from strategy_config.py. So "label equals backtest" holds exactly, and "live equals kernel" holds up to a documented list of differences in the policy_exits.py docstring. The notable ones:

- live confirms a stop breach on **two consecutive readings** before selling;
- live tracks the high-water mark on quote **midpoints**, the kernel on bar **highs**;
- in risk-off macro regimes live tightens stops by a multiplier;
- the **trailing denominator**: the kernel sets the trail distance as $\text{ATR} \times 2.0 / \text{entry price}$ and enforces HWM $\times (1 - \text{distance})$, while base_loop._desired_stop_for divides by the HWM. In the example above, with the same \$500 entry ATR, the kernel's trail is \$60,377 and the crypto live trail is $61,400 \times (1 - 1000/61,400) = \$60,400$, about \$23 (0.04%) tighter. Small, but it is a real difference between what the labels assumed and what the bot does, and stock_loop agrees with the kernel, not with base_loop. It is an open owner decision, not a bug fix anyone may make alone.

Why overlapping labels need special care

A 24-hour label computed at 14:00 and another computed at 15:00 share 23 of their 24 hours. They are not independent observations. Treating 1,000 such rows as 1,000 independent facts overstates how much you know. sample_weights.py measures each row's **average uniqueness** (how much of its label window it shares with others) and turns that into an **effective sample size**, which Chapter 6 uses to keep the validation honest.

4. What the evidence says so far

- **Labels were rebuilt exactly.** The clean rebuild re-derived the triple-barrier labels on both stores (research/campaign_2026-09_jetson/README.md §4).
- **A real label bug was fixed.** TB_Bars_fb (bars held) is a positional offset in the frame passed to the

kernel, and the `compute_tb_labels` docstring warns it becomes invalid after any row filtering. Last night's fix round moved the triple-barrier stamping to after the row filters (as-of masks, drop of incomplete rows), so the holding-period counts that feed the uniqueness weights are correct (README §3).

- **The search uses both label kinds.** In this morning's stock retrain log the trials so far draw `target_kind` both raw and tb. With 60 random start-up trials before the optimizer steers (`hypersearch_v2.PRUNE_STARTUP_TRIALS = 60`), a 70-trial run is still mostly random search, so which label "wins" tonight is not evidence of anything yet.
- **Cost-aware labels versus cost-aware scoring.** Labels are gross; the trainer's score charges `TXN_COST_PCT` per trade. SCOUT-4 found the trainer scored trades at "`pred > threshold`" while the live book trades at "`pred >= max(threshold, admission floor)`" (Chapter 2). The labels themselves are sound; the scoring had drifted from the policy, and the Phase-3 bundle now running narrows that gap.
- **The live mirror has never been reconciled on real fills in this code version.** The bots have not traded with a certified model since the rebuild, so the size of the live-versus-kernel difference in practice is unmeasured.

5. Further reading

- Marcos López de Prado, *Advances in Financial Machine Learning*, Wiley, 2018, chapter 3 (labeling and the triple-barrier method) and chapter 4 (sample weights and average uniqueness).
- Robert Pardo, *The Evaluation and Optimization of Trading Strategies*, 2nd edition, Wiley, 2008. Why a strategy must be tested with its real entry and exit rules.
- Ernest P. Chan, *Quantitative Trading: How to Build Your Own Algorithmic Trading Business*, Wiley, 2008 (2nd edition 2021). A practitioner's view of backtest and live consistency.
- J. Welles Wilder Jr., *New Concepts in Technical Trading Systems*, Trend Research, 1978. The source of the Average True Range used to size every stop here.

Chapter 5. The forecaster

Two models, one number. What the neural network and the tree ensemble each learn, how they are blended, what the "q10" veto is, and why a healthy prediction is a surprisingly small number.

1. The idea in plain words

For every symbol, every hour, the forecaster produces one number: its best guess of the percent return over the chosen horizon (12 to 48 hours; Chapter 4). A prediction of 0.35 means "I expect +0.35%." That number is the blend of two very different models:

- **An LSTM with attention** (`model_v2.RegressionLSTM`). A **neural network** is a large, flexible function with many adjustable weights that are tuned by showing it examples. An **LSTM** (Long Short-Term Memory network) is a kind of neural network built for sequences: it reads the last `seq_len` hourly rows of features one at a time, carrying a running memory forward, much as you would read a chart left to right. **Attention** is an added layer that lets the network look back over all those hours and weight the ones that matter most for this prediction.
- **A gradient-boosted tree ensemble** (`LightGBM`, `model_lgb.py`). A **decision tree** asks a series of yes/no questions ("is RSI above 62? is the 4-hour return below -1%?") and outputs an average for each final branch. **Gradient boosting** builds hundreds of small trees, each one trained to correct the errors of all the trees before it. Trees are excellent at thresholds and interactions in table-shaped data and need no scaling of inputs.

A third model rides along: a **quantile** version of the tree ensemble (the "q10" model) that predicts not the average outcome but a pessimistic one, the 10th percentile. It is used only to veto trades whose downside looks unusually bad.

2. Why it matters financially

Why blend at all

Two models that make different kinds of mistakes, averaged, usually make smaller mistakes than either alone. This is the **forecast combination** result (Timmermann, 2006): if two forecasts have errors that are not perfectly correlated, a weighted average has lower error variance. A classic finding in that literature (the "forecast combination puzzle") is that simple fixed weights such as 50/50 often beat weights estimated from data, because estimated weights are themselves noisy. This system's blend code is built around that warning.

Why predictions are tiny

New readers are often surprised that a "good" prediction is +0.2% rather than +5%. The reason is arithmetic, and it matters for every threshold in the system.

The model is trained to minimize prediction error, and the prediction that minimizes squared-style error is the **conditional average**: the average outcome among all past situations that looked like this one. Most of a price move is noise that no feature can foresee. Suppose a 24-hour return has a standard deviation of 3% and the features explain a correlation of 0.05 with it (an **information coefficient**, IC, of 0.05, which would be excellent

for hourly data). Then the best possible predictions have a standard deviation of only about

$0.05 \times 3\% = \mathbf{0.15\%}$,

and a prediction two standard deviations out, a strong signal, is about +0.30%. The forecaster is correct to be timid: large predictions would be overconfident.

Now put that next to Chapter 2's costs. The crypto admission floor is 1.20%. A healthy crypto forecaster almost never predicts a move that large, so an honest crypto model will rarely be allowed to trade at all. For stocks the floor is about 0.23%, which a strong signal can clear. This is why the threshold search, the cost floor and the prediction scale must be designed together.

3. How this system does it

The LSTM leg

`model_v2.RegressionLSTM(input_dim, hidden_dim, num_layers, dropout, n_heads)` is 42 lines:

1. a stacked `nn.LSTM` reads the window (`seq_len, features`);
2. `nn.MultiheadAttention` lets every time step attend to every other;
3. the attention output is added back to the LSTM output (a **residual** connection) and normalized (`LayerNorm`);
4. the steps are averaged (**mean pooling**) into one vector;
5. a small head (`Linear -> ReLU -> Dropout -> Linear`) outputs one number, the predicted return in percent.

The trainer (`scripts/hypermsearch_v2.py`) searches its size: `hidden_dim` 64 to 384, 1 or 2 layers, 2 or 4 attention heads, dropout 0.10 to 0.40, window `seq_len` 8 to 40 bars, learning rate, batch size, weight decay, and the horizon `forward_bars` (`adaptive_config.DEFAULT_SEARCH_SPACE`). The search uses **Optuna**, a library that tries configurations one after another and, after a start-up phase of random tries, steers toward promising regions (the TPE method). Chapter 6 explains why every one of those tries is counted against the winner.

Training details that matter:

- **Loss function.** The error measure is the **Huber loss**, which is squared error for small mistakes and absolute error for large ones, so a few wild hours cannot dominate the fit. Its switch-over point `huber_delta` is searched between 0.5 and 2.0. Each row's loss is weighted by `clamp(|y| + 1, max=50)`, so rows with big realized moves count more (`hypermsearch_v2.py`, the `torch.clamp(torch.abs(yb) + 1.0, max=50.0)` lines).
- **Scaling.** Features are scaled with a `RobustScaler` (median and interquartile range, so outliers do not distort it), fitted on each fold's **training rows only**.
- **Early stopping and a checkpoint soup.** Training stops when validation error stops improving, and the weights of the best `SOUP_K = 4` epochs are averaged into one model, which tends to generalize a little better than any single checkpoint.
- **Serving.** `predict_now.load_model` loads the network and traces it with `torch.jit.trace` for faster CPU inference on the Jetson.

The LightGBM leg

The tree model sees the same window, **flattened** into one long row: `seq_len x features` columns in a fixed order (`model_lgb.flatten_sequence`, pinned to `equal_windows.reshape(-1)` so training and serving agree). Its

defaults in `model_lgb.train_lgb`: `num_leaves = 63`, `learning_rate = 0.05`, `feature_fraction = 0.7` (each tree sees a random 70% of columns), `bagging_fraction = 0.8`, up to 500 rounds with early stopping after 20 rounds without improvement.

Because flattening multiplies columns by `seq_len`, memory on the 8 GB Jetson caps how many rows the tree leg can train on: `min(LGB_MAX_ROWS = 120,000, max(20,000, LGB_X_BYTE_BUDGET = 600 MB / row bytes))`, keeping the most recent rows. SCOUT-2 computed what that means in practice (research_signal.md C1): the crypto tree leg sees about 61% of its available training pool at short windows; the stock leg sees 84% at `seq_len` up to 18 but only 25% at 64. The "stronger learner" can end up seeing the least history, and its effective training window is set by a hyperparameter chosen for the LSTM. A lag-subset redesign is written up but not built.

The blend

Serving: `predict_now.get_live_prediction` computes both legs and calls `model_lgb.ensemble_predict(lstm_pred, lgb_pred, lstm_weight=config.get('lstm_weight', 0.6))`, i.e.

$\text{prediction} = w \times \text{LSTM} + (1 - w) \times \text{LightGBM}$, with $w = 0.6$ by default.

The default lives in one place, `blend_fit.DEFAULT_LSTM_WEIGHT = 0.6`. Under `HYPERSEARCH_V3` (switched ON this morning) the trainer also estimates w from out-of-fold predictions with `blend_fit.fit_blend_weight_v2`: a non-negative least-squares fit, a significance test that respects overlapping labels, shrinkage halfway toward the simple average, and smoothing against the previous champion's weight within $[0.25, 0.75]$

(`blend_fit.smooth_across_retrains`). Under V3 the holdout certificate is also issued on the **blended** predictor with the q10 veto applied, so "the certified predictor IS the deployed predictor" (`strategy_config.py` comment on `HYPERSEARCH_V3`). The sub-flag `BLEND_FIT_ON_REFIT = False` means the deployed weight still comes from the older of two fits; both are logged side by side.

Worked example: the LSTM predicts +0.50% and the tree model +0.20%. With $w = 0.6$ the blend is $0.6 \times 0.50 + 0.4 \times 0.20 = +0.38\%$. For a stock with a 0.23% floor and a searched threshold of 0.30%, that clears both; for crypto it is far below 1.20%.

The q10 tail veto

A **quantile regression** predicts a chosen percentile of the outcome rather than its average. The trainer fits a LightGBM model with `objective='quantile'`, `alpha=0.10` (`hypersearch_v2.train_lgb_ensemble`), so its output answers "in the worst 10% of similar situations, what return did we see?" The **floor** is the 15th percentile of that model's predictions on the validation rows (`floor = np.percentile(q10_val, 15)`), saved in `{prefix}lgb_q10_meta.json`. Live, `predict_now` puts Q10 and Q10_Floor in the snapshot, and `base_loop._execute_buys` skips any buy where $Q10 < Q10_Floor$, journaling a `q10_tail_veto`. In words: if the downside for this setup looks worse than for 85% of historical setups, pass, even if the average prediction is positive.

Worked example: the blend says +0.38%, but the q10 model says -1.40% against a floor of -1.10%. The average case looks fine; the bad case looks unusually bad. The trade is vetoed.

Serving hygiene

`predict_now` uses closed bars only. The boosters are cached by file modification time (`serving_cache.py`), fixing a past race where a new LSTM could be served with last week's trees. `prediction_cache.py` reuses a result within the same bar, so about 119 of 120 cycles per hour skip recomputation. A missing artifact means no

prediction and therefore no entry.

4. What the evidence says so far

- **The April models were not what the docs described.** The 2026-09-26 serving audit found the April artifacts served **LSTM-only**, silently: there were no tree or q10 boosters and no manifest on the box (research/campaign_2026-09_jetson/README.md §2). A full rebuild was mandatory.
- **No certified forecaster exists yet.** Crypto: 43 of 44 trials negative. Stocks: best holdout Sharpe 0.13, Deflated Sharpe 0.058 against 0.60 (README §4). The 70-trial stock retrain with the Phase-3 bundle is running as this is written.
- **Prediction scale versus floor, measured.** A Phase-3 verification slice found blended predictions peaking at only 0.64 to 0.73 (research_signal.md, SCOUT-4 T1). On crypto that means a correctly cost-anchored model will make zero trades and score 0.0, which the search treats as better than any negative score. The note flags this "abstention hazard" as an owner item.
- **Is the LSTM earning its memory?** The literature the campaign surveyed leans toward trees on hourly crypto data: Bysik and Ślepaczuk (arXiv 2606.00060, 2026) found gradient boosting "descriptively stronger" than an LSTM on hourly Bitcoin, though not with formal statistical dominance, and that the main obstacle was "the way forecasts are converted into trades." Grinsztajn, Oyallon and Varoquaux (2022) found tree models still beat deep learning on typical tabular data. PyTorch is also the largest memory item in the bot process. SCOUT-1 (S1-05) wrote a pre-registered test: if the fitted LSTM weight is not significantly above zero on two consecutive retrains and a tree-only model is no worse, propose a torch-free serving path. Nothing has been decided.
- **The q10 floor is in-sample and its coverage is untested.** The floor is computed on validation rows that, under the full-refit option, sit inside the training window (a code comment in train_lgb_ensemble says so), and "q10 holdout coverage has NO tool yet."

5. Further reading

- Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory," *Neural Computation* 9(8), 1997.
- Ashish Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems* 30 (NeurIPS), 2017.
- Guolin Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," *Advances in Neural Information Processing Systems* 30 (NeurIPS), 2017.
- Roger Koenker and Gilbert Bassett Jr., "Regression Quantiles," *Econometrica* 46(1), 1978. The idea behind the q10 model.
- Allan Timmermann, "Forecast Combinations," in *Handbook of Economic Forecasting*, Volume 1, Elsevier,
 1. Why and how to blend forecasts, and why simple weights are hard to beat.
- Léo Grinsztajn, Edouard Oyallon and Gaël Varoquaux, "Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?" NeurIPS Datasets and Benchmarks Track, 2022.

Chapter 6. Honest validation

How to tell skill from luck when you have tried many things on the same history. This is the most important chapter in Part One, and it ends with last night's verdict: no selection skill, explained kindly.

1. The idea in plain words

A model's quality can only be judged on data it has not seen. Everyone knows that. The hard part is that in finance "data it has not seen" is surprisingly difficult to arrange, for three reasons:

1. **Time leaks.** If you shuffle hourly rows randomly into a training pile and a test pile, the test row for 15:00 sits next to a training row for 14:00 that shares almost all of its future. The model has effectively seen the answer. Tests on financial data must respect time: train on the past, test on the later future.
2. **Labels leak across the boundary.** Even with a clean time split, a training row from 23:00 the night before the test period has a 24-hour label that reaches *into* the test period. That row's answer overlaps with test answers. The fix is **purging** (remove training rows whose label window crosses the boundary) and an **embargo** (skip a short gap after the boundary before testing, so slow-moving features cannot carry information across).
3. **You leak through your own choices.** Every time you try a configuration, look at its test score and try another, the test set becomes a little more like training data. After 40 tries, the best test score is partly a measure of luck. The fix is to **count your tries** and demand more of the winner the more you tried, and to keep one final slice of data, the **holdout**, that the search never touches.

A few definitions used throughout:

- **Sharpe ratio:** average return divided by the standard deviation of returns. A strategy earning 0.2% per trade with a 1% standard deviation has a **per-trade Sharpe** of 0.2. To compare across frequencies it is **annualized** by multiplying by the square root of trades (or periods) per year: $0.2 \times \sqrt{100 \text{ trades a year}} = 2.0$.
- **Walk-forward validation:** train on an early window, validate on the next window, move forward, repeat.
- **Deflated Sharpe Ratio (DSR):** the probability that the true Sharpe is above what the best of N skill-less attempts would show by luck (Bailey and López de Prado, 2014). A DSR of 0.95 is strong evidence; 0.5 means indistinguishable from luck.

2. Why it matters financially

The lucky-winner arithmetic. Take 40 "strategies" that are pure coin flips: each makes 60 trades with returns drawn from a distribution with mean zero. None has any skill. Pick the one with the best per-trade Sharpe. A simulation of that experiment (2,000 repetitions, run for this chapter with the repo's own `validation.expected_max_sharpe` as a cross-check) gives a typical best per-trade Sharpe of about **0.28**. If those 60 trades happened over a year, the annualized Sharpe is $0.28 \times \sqrt{60} = 2.2$. That is a number that would impress most investors, produced by selecting the luckiest of 40 random guesses.

The formula behind it: the expected maximum of N independent standard normal draws grows roughly like $\sqrt{2 \ln N}$. `validation.expected_max_sharpe` gives, in units of one standard deviation:

attempts N	2	10	40	100	1000
expected best	0.52	1.58	2.19	2.53	3.26

Trying 25 times more (40 to 1000) only raises the luck bar by half again, which is why counting generously costs little.

What goes wrong without it. Deploy the lucky winner and its live performance will regress toward the true value, zero, minus costs. The account pays fees for the privilege of discovering this. The 2026-07 review expected a 50 to 73% haircut from backtest Sharpe to live Sharpe even for a model that passes (`docs/MAP.md §9`).

3. How this system does it

All of this lives in `scripts/hypersearch_v2.py` (the trainer), `validation.py` (the mathematics), `sample_weights.py` (effective sample size) and `backtest.py` (the policy gate).

The holdout: the last 12%, touched once

`hypersearch_v2.get_holdout_boundary` (via `objective_utils.holdout_boundary`) sets the boundary at the 88th percentile of all row timestamps (`HOLDOUT_FRACTION = 0.12`); an option `FIXED_HOLDOUT_DAYS` pins it to a fixed number of days instead. Every later step (folds, refits, blend fit, the certificate) inherits the same boundary. The search never sees data after it.

Purged, embargoed walk-forward folds

`hypersearch_v2.get_walk_forward_folds` builds three **expanding** folds over the search region (the 88% before the holdout). Fold training windows end at the 55th, 70th and 85th percentiles of the search timestamps; each validation window covers the next 15%. Two protections:

- **Purge:** a training row is kept only if its label window *completes* before the boundary (`all_label_times <= t_train_end`).
- **Embargo:** validation starts `seq_len` bars after the boundary. With `TRAINING_REPAIRS_V1` (now ON) the gap is counted in actual bars on the data's own grid; the older rule counted calendar hours, which for stocks shrank a 40-bar embargo to about 11 trading hours.

The docstring records why folds are split by calendar time: the old code split rows by position, and since rows are stored ticker by ticker, "train on the first 60%" meant training on some coins' entire history and validating on other coins over the same dates. With cryptos correlated 0.7 to 0.9, that was "near-direct leakage."

The score the search maximizes

For each trial and fold, `hypersearch_v2.compute_sharpe` simulates non-overlapping trades (enter when the prediction exceeds the trial's threshold, hold `fb` bars) net of `TXN_COST_PCT` and annualizes by the occupied share of the year. Fewer than 10 trades scores 0.0. The trial score is

$\text{score} = \text{mean}(\text{fold Sharpes}) - 0.5 \times \text{std}(\text{fold Sharpes}),$

which rewards consistency across folds. A regime penalty then applies if any market regime's Sharpe is below -0.5. A small but telling bug: the legacy penalty multiplied the score by 0.7, which for a negative score moves it *toward zero*, rewarding a trial for doing badly in some regime. Under `TRAINING_REPAIRS_V1` the penalty is $\text{score} -= 0.3 \times |\text{score}|$, which always makes it worse.

Effective sample size

Trades that overlap in time are not independent. `sample_weights.calendar_effective_n` gives each trade a **uniqueness** u equal to the average, over the hours it was open, of $1 / (\text{number of trades open at that hour})$, and sums them: $n_{\text{eff}} = \text{sum of } u$. Worked example: three trades on three correlated coins opened in the same hour and closed together each have $u = 1/3$, so together they count as **one** effective observation, not three. Last night SIGNAL found and fixed a units bug in the holdout's n_{eff} (a seconds-versus-hours mix-up), which the campaign README lists among the night's highlights.

The Deflated Sharpe gate

`validation.dsr_from_trade_returns(trade_returns, n_trials, n_eff=...)` computes:

1. the observed per-trade Sharpe s on the holdout trades;
2. the luck bar $SR0 = \text{expected_max_sharpe}(N, 1/\sqrt{n_{\text{eff}}})$, the Sharpe the best of N skill-less configurations would show with this many effective trades;
3. $DSR = \Phi((s - SR0) \times \sqrt{n_{\text{eff}} - 1} / \sqrt{1 - \text{skew} \times s + (\text{kurt} - 1)/4 \times s^2})$, where Φ is the normal cumulative distribution and the skew/kurtosis term corrects for fat-tailed returns (`validation.deflated_sharpe_ratio`).

A model is saved only if holdout Sharpe > 0 **and** $DSR \geq DSR_{\text{MIN}} = 0.60$. The code comment is candid that 0.60 "sits well below" the conventional 0.95: it is a screen for a challenger, not a publication-grade test.

Worked example, computed with the repo's own functions: $N = 40$ trials, $n_{\text{eff}} = 60$ effective trades. The luck bar $SR0$ is **0.283** per trade. Then:

observed per-trade Sharpe	0.10	0.20	0.30	0.40
DSR	0.08	0.27	0.55	0.81

Only a per-trade Sharpe near 0.35 or above passes. `validation.min_track_record_length` turns the same idea into "how many effective trades would I need": to show a per-trade Sharpe of 0.10 is above zero at 95% confidence takes about **273** effective trades.

PBO, the ratchet and the second gate

- **PBO** (probability of backtest overfitting) asks how often the configuration that was best in one half of the data ranks below median in the other half. The search prints a coarse version from fold scores (`validation.pbo_from_fold_scores`); the full combinatorial version (`validation.pbo_cscv`, Bailey et al. 2017) is implemented and tested but has **no production caller**. `docs/MAP.md §4b` asks that anyone citing "CSCV-PBO" say so.
- **The ratchet**: the holdout is scored only when the search winner beats the stored best score (`accept_new = new_score > existing_score` in `hypersearch_v2.main`). A new study starts from a stored best of 0.0, so a search where every trial is negative never touches the holdout at all. A noisier, Thresholdout-style ratchet (Dwork et al., 2015) exists behind `PROMOTION_GATE_V2`, off.
- **The policy gate**: `backtest.py --gate` replays the saved model through the real exit kernel and costs and requires $n_{\text{trades}} \geq 10$, Sharpe > 0 and $DSR \geq 0.60$; on failure it rolls back to the previous model (`.prev` files).
- **The live test**: a retrained model goes to the **challenger** slot and runs in **shadow** (predicting, not trading) next to the **champion**. `shadow.dm_hln` compares their forecast errors with the Diebold-Mariano test (Harvey-Leybourne-Newbold correction); promotion needs at least 200 observations, 14 days and $p < 0.05$ (`shadow.MIN_OBS`, `MIN_SHADOW_DAYS`, `EARLY_PROMOTE_P`).

4. What the evidence says so far

Last night's verdict, explained kindly

Here is what happened on the clean, leak-free stores (research/campaign_2026-09_jetson/README.md §4):

- **Crypto:** 44 trials, 43 negative, one at +0.03. Its holdout produced 3 trades on 31,662 rows: "Model NOT saved: insufficient_n." The DECOMP-1 analysis found 44 of 48 fold Sharpes negative, the scoring arithmetic was not the cause, and every searched threshold was below the 1.2% admission floor, so trades lost after cost. "More trials of the same design won't help."
- **Stocks:** 25 trials, best score +0.024. Holdout: 20 trades, n_{eff} 14, Sharpe 0.13, DSR **0.058** against 0.60: "Model NOT saved: failed holdout gate." DECOMP-2 on the first 11 trials: no trial beat a zero-skill ranker by more than one standard error; a random ranker's Sharpe predicted the observed ordering of trials (rank correlation 0.65, $p = 0.03$); fold 1 was at or below zero in all 11 trials; and the study's "best" entry was the 0.0 score of a configuration that barely traded.

The kind reading is this. The system was asked "is there skill here?" and answered "not that I can detect," with a precise diagnosis of why. Nothing unskilled was saved, nothing unskilled traded, no money was lost, and the diagnosis names concrete levers (a minimum-trade rule, scoring relative to a benchmark or drift, the penalty form, conditioning on regime or entry time, thresholds anchored to cost). A validation pipeline that says "yes" to everything is worthless; this one has now proved it can say "no." That is the foundation every future "yes" will stand on.

How strict is the gate, really?

SCOUT-3 computed the gate's **power**: the chance it passes a model that truly has a given edge (research_signal.md G2). At $n_{\text{eff}} = 60$ and $N = 44$ trials, a model with a real per-trade Sharpe of 0.10 (about 2 annualized at 500 trades a year) passes only about **4.5%** of the time; at 0.20, about 18%. A skill-less model passes about 0.7% of the time. The gate is safe against false positives but "power-starved," and power falls further as the cumulative trial count grows. The note proposes that the owner pick exactly one remedy (count holdout evaluations rather than trials, lengthen the holdout, or rely on the live shadow test) and warns never to combine it with lowering DSR_MIN.

Search mechanics worth knowing

PRUNE_STARTUP_TRIALS = 60 sets both the optimizer's random start-up phase and the pruner's warm-up, so a 40-trial run is **pure random search with no pruning** despite the "TPE + pruning" banner (research_signal.md SCOUT-2, and the owner note startup_trials_owner_item.md). This morning's 70-trial stock retrain will steer only for its last 10 trials.

5. Further reading

- David H. Bailey and Marcos López de Prado, "The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting and Non-Normality," *Journal of Portfolio Management* 40(5), 2014.
- David H. Bailey, Jonathan M. Borwein, Marcos López de Prado and Qiji Jim Zhu, "The Probability of Backtest Overfitting," *Journal of Computational Finance* 20(4), 2017.
- Campbell R. Harvey and Yan Liu, "Backtesting," *Journal of Portfolio Management* 42(1), 2015. How to haircut Sharpe ratios for multiple testing.
- Andrew W. Lo, "The Statistics of Sharpe Ratios," *Financial Analysts Journal* 58(4), 2002.

- Cynthia Dwork et al., "The Reusable Holdout: Preserving Validity in Adaptive Data Analysis," *Science* 349(6248), 2015.
- Marcos López de Prado, *Advances in Financial Machine Learning*, Wiley, 2018, chapters 7 (purged cross-validation) and 11 to 14 (the dangers of backtesting).

Chapter 7. The gate stack and meta-labeling

Between "the model says buy" and "an order goes out" sit about eighteen checks. This chapter walks through them, explains meta-labeling (a second model that judges the first), and explains why some gates fail open and others fail closed.

1. The idea in plain words

A forecast is not a decision. Even a good forecaster should not act on every prediction: some are too small to pay for their costs, some arrive at moments when the market is dominated by a scheduled news event, some conflict with what the account already holds, and some come from setups where the model has historically been wrong more than it is right. A **gate** is a check that can stop a trade (a **veto**) or shrink it (a **multiplier** or **tilt**). The **gate stack** is the ordered list of those checks.

Two ideas make the stack more than a pile of rules:

- **Every veto is written down with its reason.** When a gate says no, the loop appends a skip row to the day's journal (`base_loop._journal_skip`). Later, `decision_report.py` replays what the skipped trade would have done using the same exit kernel as the backtest, so each gate's cost and benefit can be measured. A gate that blocks more winners than losers is costing money.
- **Meta-labeling.** The primary model (Chapter 5) decides *direction and size of move*. A second model, the **meta-labeler**, is trained to answer a narrower question: "given that the primary model wants to buy here, what is the probability this trade actually makes money after costs?" This idea comes from López de Prado (2018) and was formalized by Joubert (2022). It lets a simple, fairly sensitive primary signal be paired with a filter that learns when that signal tends to be wrong.

2. Why it matters financially

Loss avoidance compounds. Suppose the primary model's candidates split into two groups: 60 that average +0.30% net and 40 that average -0.50% net. Taking all 100 averages $(60 \times 0.30 - 40 \times 0.50) / 100 = -0.02\%$ per trade: a losing system. A filter that removes three-quarters of the bad group and one-sixth of the good group leaves 50 good and 10 bad, averaging $(50 \times 0.30 - 10 \times 0.50) / 60 = +0.17\%$: a winning one, from the same forecasts. That is the financial case for gates. The 2026-07 review judged that loss avoidance through the gate stack, plus cost engineering, is the most reliably valuable part of the current system (`docs/MAP.md §1`).

Over-gating is a cost too. Every gate also removes some good trades. A stack of ten gates, each blocking a "mere" 10% of good trades, passes only $0.9^{10} = 35\%$ of them. And gates that react to the same underlying condition (for example several that all respond to high volatility) can silently multiply into a near-total halt. That is why each gate is journaled and why the campaign built `scripts/sizing_cofire_report.py` to measure how often multipliers fire together.

Fail-open versus fail-closed is a financial choice. If a data feed breaks:

- failing **closed** (no trade) costs you opportunities but cannot lose money through ignorance;
- failing **open** (trade as if neutral) keeps the book running but acts without that input.

The rule here (`research/AGENT_CONTEXT.md`, convention 3; `docs/MAP.md §6`): **core inputs fail closed** (no

model, no prediction or no quote means no entry), while **optional advisors fail open** (an LLM outage or a missing meta-model must never freeze the whole book).

3. How this system does it

Book-level gates (before any symbol is examined)

`base_loop._entries_allowed` checks, in order: the manual halt file (`trading_halt.flag`); the **macro stand-down** from `macro_calendar.macro_standdown`, which blocks new entries from 12:00 to 15:30 New York time on FOMC statement days and from 06:30 to 09:30 on CPI release days (exits keep running); and a warning if the event calendar has run out. The stock book adds: already flattened today, inside an entry window (09:45 to 11:00 and 14:30 to 15:30), the SPY 200-day trend computed, and current exposure known (fail-closed). The **circuit breaker** (Chapter 8) sits above all of these.

Note a gap the docs flag honestly: there is **no stand-down for the monthly jobs report (NFP)**, though the 2026-08 literature pass recommended one (`docs/MAP.md` §9 item 14).

The per-symbol funnel

`base_loop._execute_buys` walks each candidate through this order (the stock loop iterates only its top-7 ranked names and evaluates the threshold before the cost gate, a documented difference):

#	Gate	Rule in code	Kind
1	cooldown	no re-entry within <code>cooldown_min</code> (crypto 60, stock 20 minutes)	veto
2	hard-stop lockout	24 hours after a hard-stop exit on that name (<code>lockout_hours</code>)	veto
3	daily budget	at most 4 crypto / 3 stock entries per name per day (<code>MAX_TRADES_PER_SYMBOL_PER_DAY</code>)	veto
4	position cap	already at <code>MAX_NOTIONAL_PER_SYMBOL</code>	veto
5, 6	prediction and quote present	missing means no entry	fail-closed veto
7	cost gate	<code>order_utils.should_trade</code> : prediction must beat 2x round-trip cost at the live spread (Chapter 2)	veto
8	threshold	<code>prediction >= the certified trade_threshold</code>	veto
9	winner's curse	if price > 20-bar average + 2 ATR, require 1.5x the threshold	veto
10	correlation	<code>portfolio.check_portfolio_correlation</code> against current holdings	veto
11	VIX halt	VIX above 35	veto
12	VIX-25 block	VIX above 25 blocks everything except a safe-haven list (stocks)	veto
stock	trend filter	SPY below its 200-day average blocks non-safe-havens	veto
13	sentiment	<code>sentiment.sentiment_gate</code> , clamped to [0.15, 1.5]	multiplier

#	Gate	Rule in code	Kind
			only
14	LLM	veto if score $s < 0.15$, else multiplier $0.5 + s$	veto + multiplier
15	meta-label	veto if $p < 0.30$, else multiplier $\text{clip}(2p, 0.6, 1.3)$	veto + multiplier
16	q10 tail	$Q_{10} < Q_{10_Floor}$ (Chapter 5)	veto
17	sizing	the size works out to zero (Chapter 8)	veto

The **VIX** is the Cboe Volatility Index, the market's implied expectation of S&P 500 volatility over the next 30 days, often called the "fear gauge." Readings around 12 to 20 are calm; above 30 is stress. The thresholds live in `types_mod.MacroRegime`. The list of safe havens (gold, silver, consumer staples, telecoms and similar) is `stock_config.SAFE_HAVEN_SYMBOLS`.

Meta-labeling in this system

`meta_label.py` builds its training set by **replaying** the primary model's predictions through the same exit kernel the backtest uses (Chapter 4), at **half** the live threshold (`META_THRESHOLD_FRACTION = 0.5`) so the meta-model sees many marginal trades it should learn to reject. Each replayed trade is labeled 1 if it made money net of costs, else 0. The final ~12% of rows are excluded so the policy gate stays honest. The classifier is a LightGBM model on a small feature set (`META_FEATURES`: the primary prediction, RSI, price-to-average ratio, Bollinger width, 12-hour volatility, 4- and 12-hour returns, ATR percent, sentiment, Hurst, the hour of day and a few others), and its raw score is turned into a **calibrated probability** (a number that means what it says: of trades given 0.60, about 60% should win).

Live, `base_loop._meta_gate` calls `meta_label.meta_probability_live`. If $p < \text{META_VETO_PROB} = 0.30$ the trade is skipped (`meta_veto`); otherwise `meta_label.meta_size_mult(p) = \text{clip}(2p, 0.6, 1.3)` is multiplied into the size. Worked examples:

- $p = 0.25$: vetoed.
- $p = 0.40$: allowed, size $\times 0.8$.
- $p = 0.50$: neutral, size $\times 1.0$.
- $p = 0.70$: size $\times 1.3$ (the cap; $2 \times 0.70 = 1.4$ is clipped).

If no meta-model exists or it fails to load, `meta_probability_live` returns `None` and the gate passes with a neutral 1.0: fail-open by documented intent.

The LLM gate

`llm_analyst.analyze_trades` sends one batched, schema-enforced request per cycle of `LLM_INTERVAL_SEC = 600` seconds, containing the candidates with their headlines, fundamentals, macro context and the Fear and Greed reading, and asks for a conviction score s between 0 and 1 per symbol. The LLM has exactly three powers (`docs/MODULES.md §8`):

1. **veto** a new entry when $s < \text{trading_utils.LLM_VETO_THRESHOLD} = 0.15$;
2. **force a sale** of a held position after **two consecutive** vetoing analyses (`base_loop._execute_llm_veto_sells`); two readings are required because news headlines are an untrusted channel that could carry adversarial text (**prompt injection**);

3. **tilt size** by $llm_mult = 0.5 + s$ (so $s = 0.5$ is neutral and $s = 0.8$ gives 1.3x before the overall cap).

Any failure returns an empty result, the loop falls back to the prior score or $s = 0.5$, and the trade proceeds: fail-open. Spending is capped at \$1 per day with a cross-process ledger (`llm_cost.json`). `llm_client.py` can speak to Gemini, Anthropic and OpenAI-compatible endpoints. The 2026-09-26 audit found this box configured with a Gemini key only, so the analyst runs on `gemini-2.5-flash-lite`, and a test call cost \$0.000021 (the LLM audit summarized in `research/campaign_2026-09_jetson/README.md §2`).

Fail-open versus fail-closed, collected

Fails closed (no entry)	Fails open (neutral)
missing model, prediction, or quote; stale quote over 180 s	LLM provider error ($s = 0.5$)
circuit-breaker API error (buys suspended)	meta-model absent (multiplier 1.0)
stock exposure unknown	earnings calendar and SEC EDGAR event checks for new entries
overnight sleeve without an earnings calendar	macro stand-down check on exception

4. What the evidence says so far

- **The historical journals are too old to judge the gates.** The decision journals on the box run from 2026-02-23 to 2026-05-07 and hold 11,818 skip rows, 264 buy rows and 0 sell rows in an older format without fill prices (`research_engine.md §X2(b)`). The campaign's measurement audit found that instrument verdicts on those journals are void because of schema drift (`README §2`). No gate has yet been priced on data from the current code.
- **No meta-model exists on the box**, so today the meta gate is always neutral (the device facts list "NO meta_* artifacts anywhere"). Last night meta-label training ran only its no-champion path, because there was no certified primary model to replay.
- **A meta-calibration leak was identified.** In the default ("legacy") mode the isotonic calibrator is fit on the same 20% slice the meta booster used for early stopping, so no row is held out from both (`research_signal.md SCOUT-2 C3` and `SCOUT-4 T2`). The campaign landed a measurement-only tool, `scripts/meta_calib_nested.py`, for an honest nested comparison; switching calibration mode awaits its result on real data. The meta-labeler's `pred` feature is also the primary model's in-sample fit unless `META_OOF_PRED` is enabled (default off).
- **The VIX-25 block is effectively a stock halt** in stress, because the safe-haven names are mostly not in the tradable top-7 (`docs/MAP.md §9` item 9); a flag to remove it (`VIX25_BLOCK_REMOVED`) exists, default off, pending measurement.
- **Sentiment is counted three times** (as a model feature, as a size multiplier, and inside the LLM prompt), and **VIX twice** in sizing. Both are documented as unmeasured over-counting (`docs/MAP.md §9` item 11).
- **Is the LLM worth it?** `llm_eval.py` regresses realized returns on the model's prediction and the LLM score; the keep-or-kill verdict needs at least 120 distinct hourly clusters (about 20 days of live LLM cycles). It has never been reached. An INTEL audit found the production test over-rejects at small samples and proposed an alternative estimator (`README §5`).
- **Kill list reminder:** using the shadow DM-HLN test to approve *policy* changes (such as a new gate) is killed as a category error: that test compares forecast errors, not trading rules (`research/KILL_LIST.md`).

5. Further reading

- Marcos López de Prado, *Advances in Financial Machine Learning*, Wiley, 2018, section 3.6 (meta-labeling).
- Jacques Francois Joubert, "Meta-Labeling: Theory and Framework," *Journal of Financial Data Science* 4(3), 2022.
- Michael Meyer, Illya Barziy and Jacques Francois Joubert, "Meta-Labeling: Calibration and Position Sizing," *Journal of Financial Data Science* 5(2), 2023.
- Alexandru Niculescu-Mizil and Rich Caruana, "Predicting Good Probabilities with Supervised Learning," *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 2005. Platt scaling versus isotonic calibration.
- David O. Lucca and Emanuel Moench, "The Pre-FOMC Announcement Drift," *Journal of Finance* 70(1), 1. Why scheduled Fed announcements dominate price action around them.
- Robert E. Whaley, "The Investor Fear Gauge," *Journal of Portfolio Management* 26(3), 2000. What the VIX measures.

Chapter 8. Sizing, exits and risk

How much to buy, when to get out, and the layers of limits that keep one bad day from becoming a bad year. Includes a cautionary tale about 63 trades that never happened.

1. The idea in plain words

A trading system makes three money decisions per trade, and the forecast drives only one of them:

1. **Whether** to trade (Chapters 5 to 7).
2. **How much** to trade: **position sizing**.
3. **When to get out**: the **exit rules**.

Professional traders often say sizing and exits matter more than entries, and the arithmetic backs them: a mediocre entry with disciplined sizing survives; a brilliant entry with reckless sizing can wipe out an account on one unlucky sequence.

Terms used below:

- **Stop-loss (stop)**: a price below the entry at which you sell to cap the loss.
- **Take-profit (TP)**: a price above the entry at which you sell to lock in a gain.
- **Trailing stop**: a stop that moves up as the price rises (it trails the **high-water mark**, the highest price since entry), but never moves down.
- **Drawdown**: the percentage fall from the account's highest value to its current value.
- **Kelly criterion**: a formula for the fraction of capital to bet that maximizes long-run growth, given your win probability and payoff.
- **Volatility targeting**: sizing so that each position contributes roughly the same amount of expected fluctuation, smaller in wild assets and larger in calm ones.

2. Why it matters financially

Losses are asymmetric. Lose 10% and you need +11% to recover; lose 25%, +33%; lose 50%, +100%. Deep drawdowns are disproportionately hard to climb out of, so a rule that cuts size as drawdown grows protects the account's ability to recover.

Kelly and why nobody bets full Kelly. For a bet that wins with probability p and pays b times the amount lost when it loses, the Kelly fraction is

$$f^* = p - (1 - p) / b.$$

With $p = 0.55$ and $b = 1.2$ (average win 1.2%, average loss 1.0%): $f^* = 0.55 - 0.45 / 1.2 = \mathbf{0.175}$, meaning risk 17.5% of capital per bet. Two facts make that dangerous in practice. First, full Kelly has brutal swings: a well-known property is that it has about a one-in-two chance of halving the account at some point along the way. Second, p and b are *estimated*, and if your true edge is half what you estimated, betting "full Kelly" on the estimate means betting twice the true Kelly, where long-run growth falls to about zero. Hence the practitioner's habit of **fractional Kelly** (a half or a quarter), which gives up a little growth for much smaller drawdowns (MacLean, Thorp and Ziemba, 2011).

Stops must be sized to the asset. A fixed 2% stop is too tight for a coin that routinely moves 2% in an hour (you get stopped out by noise) and too loose for a sleepy utility stock. Scaling stops by **ATR** (Chapter 4) adapts to each asset's normal range.

3. How this system does it

Sizing: `base_loop._compute_position_size`

One function sizes every entry in both books. Its docstring says it replaced an "unbounded multiplier soup" that could stack to about 32 times the base size. Each component now has one job and a hard bound.

Step 1, risk base. Risk a fixed fraction of equity to the stop:

`notional = equity x RISK_PCT_PER_TRADE (0.005) / stop distance,`

capped at `NOTIONAL_PER_SYMBOL` (\$1,000 for crypto from the base loop, \$5,000 for stocks from `stock_loop.StockLoop`). The stop distance is `ATR x the policy multiple / price`, clipped to the policy floor and ceiling (Chapter 4).

Step 2, Kelly multiplier. `trading_utils.compute_kelly_fraction` reads the book's confirmed trades from `trade_memory.json`, needs at least 50, uses the most recent 200, **shrinks** the win rate and payoff toward a skeptical prior of 50 pseudo-trades at breakeven (win rate 0.5, payoff 1.0), computes Kelly, halves it, and clamps to `[0.05, 0.25]` (`KELLY_CAP = 0.25`). The loop maps 0.125 to a 1.0x multiplier and bounds the result to `[0.5, 1.5]`. Worked example with 100 trades at 55% wins and a 1.2 payoff: shrunk win rate $(55 + 25) / 150 = 0.533$, shrunk payoff $(120 + 50) / 150 = 1.133$, Kelly 0.122, half-Kelly 0.061, multiplier $0.061 / 0.125 = 0.49$, clamped to **0.5x**. Notice that a respectable track record maps to the *minimum* multiplier: reaching 1.0x requires a half-Kelly of 0.125, i.e. a full Kelly of 0.25, which is a very strong edge. With fewer than 50 trades the function returns None and the multiplier is neutral 1.0. When VIX is above 25 or the HMM regime model says "bear," the Kelly multiplier is capped at 1.0.

Step 3, volatility target. `volatility.compute_vol_adjusted_size` compares the asset's forecast hourly volatility (HAR-RV on intraday ranges, with GARCH as fallback; `volatility.get_sigma`, output in decimal per bar) with the book's target converted to hourly units: `PORTFOLIO_VOL_TARGET = 35%` a year for crypto and 18% for stocks, so the crypto target is $0.35 / \sqrt{8760} = 0.374\%$ per hour. The ratio target / forecast is clamped to `[0.5, 1.5]`. Worked example: SOL forecast at 0.90% per hour gives $0.374 / 0.90 = 0.42$, clamped to **0.5x**.

Step 4, the tilt product. Every advisory input multiplies into one number (detail in the sizing journal records each factor):

Factor	Rule
signal confidence	prediction / threshold, clipped to <code>[0.75, 1.25]</code>
VIX ladder	1.0 up to 15, 0.7 above 15, 0.5 above 25, 0.3 above 35
drawdown ladder	0.75 at 10% drawdown, 0.50 at 15%, 0.25 at 20% (<code>drawdown.DRAWDOWN_LADDER</code>)
macro regime	<code>macro_regime.sizing_mult</code> (stress, stablecoin peg, SPY trend)
correlation	haircut if correlated with current holdings
HMM regime	1.2 bull, 0.3 bear, 0.5 high-volatility (<code>regime_detector.py</code>)

Factor	Rule
disagreement	0.8 if the macro and HMM "votes" disagree
sentiment, LLM, meta	from Chapter 7
book-specific	crypto: the perpetual-funding crowding tilt (<code>crypto_loop._extra_tilt</code>)
book volatility	account-level realized-volatility scalar (<code>portfolio.get_book_vol_scalar_cached</code>)

The product is clamped to $[0.1, \text{TILT_MAX} = 1.30]$: boosts are capped at 1.3x, but de-risking is honored down to 0.1x, so a drawdown ladder that says "cut 75%" is not overridden. If two or more advisory inputs are missing (VIX, return history, correlation matrix, Kelly history), the tilt is capped at 0.5 ("degraded mode"). An emergency (for example a stablecoin losing its peg sets the macro multiplier to 0) returns size 0 before any of this.

Step 5, the book risk budget. With other positions open, the loop computes how much more stop-risk the book can take under an equal-correlation model (`portfolio.book_risk_budget`, enforced in `base_loop`, cap `MAX_BOOK_RISK_PCT = 0.025`). The formula solves for the new risk r that keeps $\sqrt{(1 - \rho) \times \sum(r_i^2) + \rho \times (\sum r_i)^2}$ at the cap. Worked example: two open positions each risking 0.5% of equity, average correlation 0.5: the new position may risk up to about **1.90%**, far above the 0.5% base, so the budget does not bind here; with six correlated positions it would.

Step 6, hard caps. The order is limited by the remaining room under `MAX_NOTIONAL_PER_SYMBOL` (\$3,000 crypto, \$5,000 stocks), divided by the leverage of leveraged ETFs, and dropped if below `MIN_ORDER_NOTIONAL = $100` ("fees eat dust").

A full worked example (stocks). Equity \$122,202. A \$100 stock with an entry ATR of \$0.60.

1. Stop distance $2.0 \times 0.60 / 100 = 1.2\%$; risk notional $\$611 / 0.012 = \$50,917$; capped base **\$5,000**.
2. Kelly: no confirmed history yet, so 1.0.
3. Volatility: forecast 0.50% per hour against $0.18 / \sqrt{1638} = 0.445\%$, ratio **0.89**.
4. Tilt: confidence $0.40 / 0.30 = 1.33$, clipped to 1.25; VIX 18 gives 0.7; no drawdown 1.0; HMM bull 1.2; LLM $s = 0.6$ gives 1.1; everything else 1.0. Product $1.25 \times 0.7 \times 1.2 \times 1.1 = \mathbf{1.155}$.
5. Sized: $\$5,000 \times 1.0 \times 0.89 \times 1.155 = \$5,140$.
6. Cap: room under \$5,000 is \$5,000. **Final order: \$5,000**, risking \$60 (0.05% of equity).

The caps did almost all the work, which is why `docs/MAP.md` §5-10 says the book runs at roughly a third to a fifth of its configured risk.

Exits

- **Stops, trailing and take-profit** follow the policy in Chapter 4: for stocks, stop 2.0 ATR, TP at 2 times the stop distance, trail 2.0 ATR arming at +1%. For the example: stop \$98.80, TP \$102.40, trail arms at \$101.00.
- **Protection at the broker.** Crypto positions carry a resting good-till-cancelled stop-limit order at Alpaca, which survives if the bot crashes; stock entries are **bracket** orders whose stop leg is upgraded to a native `trailing_stop` once the trail arms (`stock_loop._manage_stops`).
- **Local exit loop.** `base_loop._manage_stops` runs first in every cycle: hard stop, take-profit, trailing, requiring **two consecutive readings** to confirm a breach so one bad quote cannot trigger a sale. Signal sells fire when the prediction turns sufficiently negative (`_execute_sells`); stocks also sell a holding that drops out of the top 15 while its prediction is negative.
- **End of day.** Stocks are flattened within about 10 minutes of the close, except up to two overnight-sleeve positions of at most 5% of equity each that are still predicted up (`OVERNIGHT_SLEEVE_*`); the sleeve fails

closed without a live earnings calendar.

Account-level brakes

- **Circuit breaker** (`base_loop._circuit_breaker_check`): if the account is down 5% on the day against Alpaca's `last_equity` baseline (`CIRCUIT_BREAKER_PCT = 0.05`), the bot flattens its book, sends an alert, and halts entries until the baseline resets. If the risk state cannot be read, buys are suspended for that cycle (fail-closed).
- **Drawdown ladder** persistence: `drawdown.restore_peak_equity` keeps the high-water mark across restarts. Before this fix a restart mid-drawdown reset the peak and silently disabled the ladder exactly when the account was underwater (`drawdown.py` docstring, wave-8 #4).
- **Cooldowns and lockouts**: see the funnel in Chapter 7.
- **Cross-book cap**: `risk_budget.ACCOUNT_RISK_CAP = 0.03` is currently **journalled, not enforced** (`_record_account_risk`); the per-book 2.5% cap is the enforced one (`docs/MAP.md` §9 item 10).

4. What the evidence says so far

- **The phantom-trade Kelly problem.** The most instructive finding of the night (ENGINE R6 and R7, `research/campaign_2026-09_jetson/CHANGELOG.md`). In April 2026 a bug made the crypto bot believe positions had vanished ("DESYNC"), and it bought again and again: 67 false "Position gone" lines, \$95.8k of buys and no sells. The trade memory recorded **63 phantom broker_stop exits** that never happened, and none is marked estimated, so Kelly counts them. The result: crypto Kelly computes to 0.1995 on 73 rows, 63 of them phantom, which maps to a **1.5x** crypto sizing multiplier today. A repair list exists but was not applied, and the sample gate that would hold Kelly neutral (`KELLY_SAMPLE_GATE`) is off. The lesson: a sizing rule that learns from its own history is only as honest as that history. With the crypto book now on hold and the account liquidated, the exposure is moot for the moment, but it must be repaired before crypto trades again.
- **Broker quirks touch risk too.** Six inherited crypto positions had zero cost basis on an asset id none of the bot's 1,320 orders had used, so it was unverified whether the resting stops protected them at all. The founder liquidated the paper account to cash this morning, which removed the question (README §6).
- **Double counting in sizing.** VIX enters both the ladder and the macro multiplier; volatility enters the ATR base, the vol target, the VIX ladder, the HMM and the book scalar. A default-off alternative composition (`DERISK_STACK_V2`, which takes the minimum within a family of regime signals instead of the product) is computed and journalled on every entry as a shadow, so the two can be compared on real fills before anyone flips it.
- **Alerting was silent.** No Telegram or webhook was configured, so every breaker, crash or halt alert before last night went nowhere (README §6 item 4).
- **Kill list:** strategy-level volatility targeting as a source of *alpha* is killed; forecasting volatility for *sizing* (HAR-RV) survives and is live (`research/KILL_LIST.md`, commonly confused survivor #8).

5. Further reading

- John L. Kelly Jr., "A New Interpretation of Information Rate," *Bell System Technical Journal* 35(4), 1956.
- Edward O. Thorp, "The Kelly Criterion in Blackjack, Sports Betting and the Stock Market," in *Handbook of Asset and Liability Management*, Volume 1, Elsevier, 2006.
- Leonard C. MacLean, Edward O. Thorp and William T. Ziemba (eds.), *The Kelly Capital Growth Investment Criterion: Theory and Practice*, World Scientific, 2011.

- Sanford J. Grossman and Zhongquan Zhou, "Optimal Investment Strategies for Controlling Drawdowns," *Mathematical Finance* 3(3), 1993.
- Alan Moreira and Tyler Muir, "Volatility-Managed Portfolios," *Journal of Finance* 72(4), 2017.
- Fulvio Corsi, "A Simple Approximate Long-Memory Model of Realized Volatility," *Journal of Financial Econometrics* 7(2), 2009. The HAR-RV model behind `volatility.py`.

Chapter 9. The life of a trading day

9.1 The idea

A trading system is not a prediction. It is a loop. Every half minute it wakes up, looks at the world, decides whether anything must be done, does it, writes down what it did and why, and goes back to sleep. Everything interesting about the system (the model, the gates, the risk limits) lives inside that loop, and every weakness of the system shows up there too: a step that runs in the wrong order, a quote that arrives late, a decision that nobody recorded.

This chapter follows one day of that loop. It is the most concrete chapter in the book, and it is worth reading slowly, because every later chapter (training, measurement, the LLM, the hardware) is ultimately about making this loop do the right thing or about checking whether it did.

Three terms first.

- A **book** is one family of positions managed by one loop. This system has two: the **crypto book** (six coins, trades 24 hours a day, 7 days a week) and the **stock book** (a universe of US stocks, trades only while the exchange is open). Since the morning of 2026-09-27 only the stock book runs; section 9.5 explains why.
- A **cycle** is one pass through the loop.
- A **journal** is the append-only diary the loop writes: one line of JSON for every decision, including the decisions *not* to trade.

9.2 Why it matters financially

Money is made or lost at specific moments, and the loop decides which moments it participates in. Four financial facts shape the day.

1. **Costs are not constant through the day.** Spreads (the gap between the best price a buyer offers and the best price a seller asks) are widest in the first minutes after the open, when overnight news is still being digested. A strategy that trades at 9:31 pays more than the same strategy at 10:15. Gao, Han, Li and Zhou (2018) also found that intraday predictability in the US market concentrates in the first and last half hours; the middle of the day is mostly noise that still charges you a spread.
2. **The night is different from the day.** Lou, Polk and Skouras (2019) showed that a large share of the stock market's long-run return is earned overnight, between the close and the next open, while prices can also gap overnight past any stop you set. Holding overnight is both where return lives and where unprotected risk lives.
3. **Scheduled news moves markets in predictable windows.** Federal Reserve (FOMC) statements and CPI inflation prints produce some of the largest moves of the year, and a model trained on hourly bars has no special insight into them. Lucca and Moench (2015) documented the unusual behaviour of stocks around FOMC announcements.
4. **An exit that is late is an exit that failed.** A stop-loss that is checked after a slow housekeeping task, or not checked because a price quote is stale, is protection on paper only.

The loop's structure is an answer to these four facts.

9.3 How this system does it

The process tree

On the Jetson, a systemd user service (`~/ .config/systemd/user/trader.service`, generated by `scripts/setup_jetson_system.sh --user`) starts `run_pipeline.py` with `--combined-bots --bot-only`. A local override file adds `--stock-only`. The pipeline launches `run_bots.py`, which runs each book's loop as a thread in one process (this saves roughly half a gigabyte of memory compared with one process per book; Chapter 13 explains why that matters). Each loop is an instance of `BaseTradingLoop` in `base_loop.py`, specialised by `crypto_loop.py` or `stock_loop.py`. This is the "template method" pattern: the base class fixes the order of the steps, and each book fills in the details that differ (market hours, how a buy order is placed, how the day ends).

Starting up

`BaseTradingLoop.run` does four things before its first cycle:

1. Starts the optional order stream (only if `TRADER_ORDER_STREAM=1`).
2. Loads the model with `_load_models`. If any model file is missing, the model is `None`, and every buy decision later **fails closed**: no prediction means no entry.
3. Cancels stale open orders, but only for this book's own symbols, so one book never strips the other book's protective stops.
4. Rebuilds its positions with `_reconstruct_positions` from `stock_position_state.json` (or the crypto equivalent): high-water marks, trailing-stop state, cooldowns, the daily trade budget and the peak equity used by the drawdown ladder.

Then it enters `while True: self._run_one_cycle()`. An exception inside a cycle is logged and alerted, and the loop sleeps and continues. The design comment explains why: a crash-restart cancels working orders and wipes in-memory state, which is worse than skipping one beat.

One cycle, in order

The order below is the order in `BaseTradingLoop._run_one_cycle`. It is deliberate.

Step	What happens	Function
1	Remote flatten request (a file such as <code>flatten_stock.flag</code> , or a Telegram command)	<code>_check_flatten_request</code>
2	Is the market open? Crypto: always. Stocks: the Alpaca market clock, which knows holidays and half days; a wall-clock fallback (9:30 to 16:00 ET, weekdays) if the clock call fails	<code>check_market_hours</code>
3	Circuit breaker: if the account is down 5 % on the day (<code>CIRCUIT_BREAKER_PCT = 0.05</code>), flatten and halt entries	<code>_circuit_breaker_check</code>
4	End of day (stocks only): in the last 10 minutes before the actual close, flatten, keeping a small overnight sleeve	<code>flatten_before_close</code>
5	Exits : hard stop, take-profit, trailing stop	<code>_manage_stops</code>
6	Hot reload if a new model was promoted	<code>_hot_reload_check</code>

Step	What happens	Function
7	Every 10th cycle: macro regime, equity, correlations, account risk	<code>_update_macro_regime</code> and siblings
8	Predictions for every symbol, with a 45 second hard deadline	<code>_get_predictions</code>
9	Hourly challenger "shadow" predictions (no trading effect)	<code>shadow.maybe_log_shadow</code>
10	LLM analysis, at most every 600 seconds (<code>LLM_INTERVAL_SEC = 600</code>)	<code>_run_llm_analysis</code>
11	Signal sells, then LLM veto sells	<code>_execute_sells</code> , <code>_execute_llm_veto_sells</code>
12	Entries , only if buys are allowed	<code>_execute_buys</code>
13	One <code>cycle_latency</code> journal row with the time spent in each phase	<code>_journal_cycle_latency</code>
14	Sleep	<code>_sleep</code>

Notice that exits (step 5) run before the hot reload and the maintenance block. The code comment cites the reason: rebuilding the correlation matrix alone took 11 to 28 seconds, and a protective exit must never wait behind housekeeping.

The sleep in step 14 is `LOOP_INTERVAL = 30` seconds plus a random jitter of up to 5 seconds either way, never less than 10 seconds. If the GPU is hotter than `THERMAL_THROTTLE_TEMP = 75 °C` (defined in `trading_utils.py`) the interval doubles. So "a 30 second cycle" really means "do the work, then rest about 30 seconds". A slow cycle stretches the gap.

The gates in front of every entry

Before the per-symbol checks, the whole book must be allowed to enter (`_entries_allowed`):

- no manual halt (the file `trading_halt.flag`);
- no scheduled macro stand-down: FOMC statement days from 12:00 to 15:30 ET and CPI days from 06:30 to 09:30 ET (the `_windows` table in `macro_calendar.py`);
- for stocks, additionally: the book was not already flattened today, and the clock is inside an **entry window**.

The entry windows live in `strategy_config.py`: `STOCK_ENTRY_WINDOWS_ET = [('09:45', '11:00'), ('14:30', '15:30')]` with `ENTRY_WINDOWS_ENABLED = True`. The comment explains 9:45 rather than 9:30: the first fifteen minutes are dominated by the opening auction unwinding, with wide spreads and quote-driven adverse selection. Only *entries* are windowed. Exits and stops run all day.

Then each candidate symbol walks a funnel. The full table is in `docs/MAP.md` section 4e; the shape of it is: cooldown since the last trade in that name, a 24 hour lockout after a hard stop, a daily trade budget per symbol, a position cap, a prediction and a fresh quote must both exist (fail closed), the **cost gate** (the prediction must beat the round-trip cost including the quoted spread), the model's certified threshold, a "winner's curse" check for names already stretched far above their average, correlation with existing positions, VIX limits, the SPY 200-day trend filter for stocks, the sentiment multiplier, the LLM veto, the meta-label veto, a lower-tail (q10) veto, and finally sizing. The stock book only considers its top `TOP_N = 7` names by prediction rank, and keeps holding a name while it stays within `HOLD_RANK = 15`. That gap between the entry rank and the hold rank is a deliberate "buy/hold spread": Novy-Marx and Velikov (2016) found it to be the most effective way to stop a strategy from paying costs to churn names that hover around a cutoff.

Sizing

Position size is risk-based (`_compute_position_size`): the amount you are willing to lose if the stop is hit, $RISK_PCT_PER_TRADE = 0.005$ (half a percent of equity), divided by the distance to the stop. That base is then capped by the per-symbol notional limit and multiplied by a clamped product of "tilts" (Kelly, volatility target, VIX, drawdown, sentiment, LLM, meta-label and others), and finally limited by a book-wide risk budget, $MAX_BOOK_RISK_PCT = 0.025$.

The end of the stock day

`StockLoop.flatten_before_close` runs in the last 10 minutes before the actual close according to the Alpaca clock (so a half day before a holiday closes early and the flatten moves with it; the wall-clock fallback is 15:50 ET). It sells everything except the **overnight sleeve**, chosen by `_select_overnight_keepers`:

- `OVERNIGHT_SLEEVE_ENABLED = True`, at most `OVERNIGHT_SLEEVE_MAX_POSITIONS = 2` names;
- each kept position at most `OVERNIGHT_SLEEVE_MAX_PCT_EQUITY = 0.05` of equity;
- the model must still predict at least `OVERNIGHT_SLEEVE_MIN_PRED = 0.0`;
- no leveraged ETFs, no name with an earnings report ahead, and if the earnings calendar is unavailable the sleeve keeps **nothing** (fail closed, because a stop cannot protect against an earnings gap);
- no "bounced loser" (a stock that fell during the day and jumped into the close, a pattern that tends to reverse by the next open);
- the survivors are ranked 70 % by model prediction and 30 % by overnight momentum, and keepers get a good-till-cancelled (GTC) stop before the day orders expire.

It also sells **orphans**: broker positions in the universe that the loop lost track of, so that nothing unmanaged rides overnight.

The crypto book never closes; its protection overnight is a resting GTC stop-limit order at the broker, with the limit 2 % below the trigger (`RESTING_STOP_LIMIT_GAP = 0.02` in `crypto_loop.py`).

What the journals record

`trade_journal.py` writes one file per day, `journals/YYYY-MM-DD.jsonl`. Its docstring is the contract for every row type: buy (prediction, sentiment, LLM score and multiplier, final notional, decision and fill prices, slippage, the full sizing decomposition), sell (exit reason, P&L), skip (which gate said no, plus the prediction it overruled), `llm_analysis` (the scores the LLM gave and what they cost), `llm_backoff` and `llm_error`, `entry_window`, `account_risk`, and `cycle_latency`. Rotation is off by default (`TRADER_JOURNAL_ROTATE_DAYS=0`). The reason every *skip* is recorded is the subject of Chapter 11: a veto is a counterfactual trade, and you can only price a gate if you know what it blocked.

9.4 A worked example

A stock day, clock in hand. Take an ordinary Tuesday with no FOMC or CPI release.

- 09:30 ET. The Alpaca clock reports the market open. The loop runs full cycles: exits are managed, predictions are computed, but `_in_entry_window` returns `False`, so no buys.
- 09:45. The first entry window opens. Suppose the model ranks a stock third, its prediction clears the cost gate and the certified threshold, and every other gate passes.
- Sizing: equity is \$100,000, so the risk budget is $\$100,000 \times 0.005 = \500 . The stop sits at $2 \times ATR$ (the stock policy's `atr_stop_mult`); if the hourly ATR is 1.2 % of price, the stop distance is 2.4 %, and $\$500 /$

0.024 = \$20,833 of stock. But the stock book's per-symbol cap is \$5,000 (MAX_NOTIONAL_PER_SYMBOL in stock_loop.py), so the cap binds and the real risk is $\$5,000 \times 2.4\% = \120 , about 0.12 % of equity. The lesson: in this system the caps, not the elegant formula, usually decide the size. Any claim about "risk per trade" should be checked against the cap.

- 11:00 to 14:30. No new entries. Stops, trailing stops and signal exits keep running.
- 14:30 to 15:30. The second window.
- About 15:50 (ten minutes before the close). The flatten runs. Up to two positions that meet the sleeve rules are kept with GTC stops; everything else is sold.

A real cycle. The first cycle of the crypto loop on 2026-09-27 wrote this cycle_latency row: exits 6.1 s, maintenance 1.9 s, predictions 0.3 s, LLM 30.9 s, total 39.2 s. The LLM call hit its 30 second latency ceiling. That cycle therefore took 39 seconds of work *plus* the 30 second sleep, so the next cycle began more than a minute later. Nothing broke, but it shows why the exits run first: had they run after the LLM, a stop would have been checked 30 seconds late.

A journal that tells a story. The journal for 2026-04-20 contains 1,110 rows, and every one of them is a skip whose reason reads like `llm_below_buy_min (0.58<0.60)`. In April the code had a rule that required the LLM score to be at least 0.60 before buying; that rule has since been removed. Those journals describe a *different policy* from today's, which is why the measurement tools in Chapter 11 exclude them.

9.5 What the evidence says

Be candid about what is known.

- **The design choices are literature-backed, not locally proven.** The entry windows, the overnight sleeve and the macro stand-downs each cite a published result (Gao et al. 2018; Lou, Polk and Skouras 2019; Lucca and Moench 2015). None of them has been measured on this system's own trades, because the current code has not traded with a model. The bots did not run from 2026-05-07 until the overnight campaign of 2026-09-26/27, and in that campaign no model passed certification (Chapter 14), so every buy failed closed.
- **The loop behaved as designed when it did run.** From 03:09 on 2026-09-27 the bots ran in an exits-only mode against six inherited crypto positions. The ENGINE department had written a fake-broker harness and predicted what the loop would do; the live journal matched every observable prediction (six reconstruct warnings, six GTC stops placed, no spurious exits).
- **Stale quotes silently skip exit checks.** A crypto quote older than 180 seconds is treated as missing, and a missing quote means no software exit check for that name in that cycle. A census found ETH quotes stale in about 15 % of cycles over ten minutes and SOL quotes older than 180 s in 18 of 24 polls on a quiet Sunday night. The candidate fix (CRYPTO_QUOTE_MAX_AGE_SEC, default off) waits on an owner decision (ENGINE owner items [\[research/campaign_2026-09_jetson/session_artifacts/generals/engine/OWNER_ITEMS.md\]](#), item B.1).
- **The book is currently stock-only.** On 2026-09-27 the founder directed the effort toward stocks; the service now runs --stock-only and the paper account was liquidated to cash. The crypto code is intact and can be switched back on.

9.6 Further reading

- Larry Harris, *Trading and Exchanges: Market Microstructure for Practitioners*, Oxford University Press, 2003. What a spread, a limit order and a stop order really are.
- Lei Gao, Yufeng Han, Sophia Zhengzi Li and Guofu Zhou, "Market Intraday Momentum," *Journal of*

Financial Economics 129(2), 2018, 394 to 414. The basis for the entry windows.

- Dong Lou, Christopher Polk and Spyros Skouras, "A Tug of War: Overnight versus Intraday Expected Returns," *Journal of Financial Economics* 134(1), 2019, 192 to 213. The basis for the sleeve.
- David Lucca and Emanuel Moench, "The Pre-FOMC Announcement Drift," *Journal of Finance* 70(1), 2015, 329 to 371. Why FOMC days are special.
- Robert Novy-Marx and Mihail Velikov, "A Taxonomy of Anomalies and Their Trading Costs," *Review of Financial Studies* 29(1), 2016, 104 to 147. Buy/hold spreads as cost control.
- In this repository: docs/MAP.md section 4e (the full cycle and funnel tables) and docs/STATE_FILES.md section 5 (the journals).

Chapter 10. From data to a deployed model

10.1 The idea

A model does not deserve money because it was trained. It deserves money only after it has survived a sequence of tests, each designed to catch a different way that a model can look good without being good. This chapter walks the whole path a model takes in this system:

harvest (build the data) → **train** (search for a model) → **certify** (test it once on data it never saw) → **gate** (replay the real trading policy) → **champion or challenger** (decide which slot it occupies) → **shadow** (let it predict silently next to the current model on live data) → **promote** (swap it in) → **hot reload** (the running bots pick it up).

Two vocabulary words carry the chapter.

- The **champion** is the model the bots trade with. Its files are named `model_v2.*` for crypto and `stock_model_v2.*` for stocks.
- A **challenger** is a newer model waiting to prove itself. Its files carry a `challenger_` prefix. It trades nothing. It only predicts, and its predictions are compared with the champion's.

10.2 Why it matters financially

The single most expensive mistake in systematic trading is deploying a model whose backtest was an accident. If you try many model configurations and keep the best, the best will look impressive even when none of them has any skill, simply because you picked the luckiest. This is **selection bias**, and it is the reason most published trading strategies decay after publication (McLean and Pontiff 2016 measured a 58 % drop in returns after publication, of which a meaningful part is data mining).

Every stage in this pipeline exists to shrink that risk, and each one costs something: time, data, or false rejections of models that were actually fine. A good pipeline accepts that it will sometimes refuse a good model, because the alternative (sometimes accepting a lucky bad one) loses real money.

10.3 How this system does it

Stage 1: Harvest

`scripts/harvest_crypto_data.py` and `scripts/harvest_stock_data.py` build the two **feature stores**, `training_data.parquet` and `stock_training_data.parquet`. For each book they fetch hourly price bars, compute features with **the same functions the live loop calls** (`indicators.compute_features` and `compute_stock_features`), merge point-in-time external data (funding rates and open interest for crypto, short-sale volume for stocks), and apply an as-of universe mask for stocks: a name is in the universe on a given date only if it was among the top 60 by 30-day dollar volume *on that date*. That mask is where survivorship bias is removed (Chapter 15 explains the trap).

Then come the **labels**, the answers the model is trained to predict: the return from holding exactly `fb` bars, and a "triple-barrier" return from `policy_exits.compute_tb_labels`, which walks each hypothetical trade through the same exit rules (stop, take-profit, trailing stop, time limit) that the backtester and the live loop use. Using one exit

kernel everywhere is what lets the documentation say "labels equal backtest".

The clean rebuild on 2026-09-26 produced a crypto store of 263,889 rows by 71 columns and a stock store of 1,536,356 rows by 110 columns (92 names, all bars from Alpaca). It also fixed a real look-ahead leak: the `Daily_Sentiment` column had been dated so that each row held the next day's value. Every store built before that date carried the leak.

Stage 2: Train

`scripts/hypersearch_v2.py` runs an Optuna search. Each **trial** picks a configuration (forecast horizon `forward_bars`, input window `seq_len`, network size, dropout, learning rate, the trade threshold, and whether to use the raw or triple-barrier target) and trains a `RegressionLSTM` (a small recurrent neural network) on three **walk-forward folds**: train on the past, validate on the period right after, then roll forward. Between train and validation it **purges** rows whose label window overlaps the validation period and adds an **embargo** gap, so information cannot leak across the boundary.

The trial's score is computed in `_train_walk_forward` as

```
score = mean(fold Sharpe) - 0.5 x std(fold Sharpe)
```

where each fold Sharpe comes from simulating non-overlapping trades net of costs (`compute_sharpe`). A model that does well in one fold and badly in another is penalised for the inconsistency. `compute_sharpe` returns exactly 0.0 when a fold produces fewer than 10 trades, a detail that matters in Chapter 14.

The first `PRUNE_STARTUP_TRIALS = 60` trials are effectively random search: Optuna's TPE sampler needs a warm-up before it starts steering, and this constant sets both the sampler's and the pruner's warm-up. A 25 or 40 trial run is therefore pure random search.

Stage 3: Certify on the holdout

Before the search starts, the last slice of time (by default about 12 % of timestamps, `objective_utils.holdout_boundary`) is set aside as a **holdout**. The search never sees it. After the search, the winning configuration is retrained and evaluated once on the holdout. The certificate records the holdout Sharpe, the number of trades, the **effective** number of independent trades (`n_eff`, which is lower than the raw count when trades overlap in time), and the **Deflated Sharpe Ratio** (DSR).

The DSR, from Bailey and López de Prado (2014), answers one question: given how many configurations were tried, what is the probability that this Sharpe ratio reflects real skill rather than the luck of picking the best of many? It compares the observed Sharpe with the Sharpe you would *expect* from the best of N skill-less trials. The model is saved only if (the `gate_ok` expression in `hypersearch_v2.py`)

```
holdout Sharpe > 0 and DSR >= DSR_MIN (0.60, in validation.py)
```

Otherwise the log prints `Model NOT saved: failed holdout gate` and, pointedly, "A higher in-search score that cannot clear unseen data is selection bias, not skill."

Stage 4: The policy gate

The holdout test uses a simplified trade simulation. The **policy gate**, `backtest.py --gate`, replays the saved model through the real trading policy: the same exit kernel, the two-tier cost model, the meta-label veto and the `q10 veto`. `run_pipeline.py` runs it over the last 44 days for crypto (inside the holdout) and 60 days for stocks. It passes only if `n_trades >= 10`, `Sharpe > 0` and `DSR >= 0.60`. A failure exits with code 3, a deterministic

rejection that the pipeline never retries; for the champion slot it restores the previous model from its `.prev` backup files.

Stage 5: Champion, challenger, shadow

If a champion already exists, a retrain saves its result into the challenger slot instead (`--shadow`, controlled by `TRADER_SHADOW_MODE`, default on). The bots then log, once an hour, both models' predictions for every symbol into `{prefix}shadow_preds.jsonl` (`shadow.maybe_log_shadow`, every `SHADOW_LOG_INTERVAL_SEC = 3590` seconds).

Once a day, `shadow.py evaluate_and_maybe_promote` waits until each logged prediction's forecast horizon has passed, computes each model's squared error, and runs a **Diebold-Mariano test with the Harvey-Leybourne-Newbold correction** (`dm_hln`). The DM test asks whether one forecaster's errors are smaller than another's by more than chance; the HLN correction makes it behave better in small samples. The legacy rule, in constants at the top of `shadow.py`:

- promote early if at least `MIN_OBS = 200` paired observations, at least `MIN_SHADOW_DAYS = 14` days old, and $p < 0.05$;
- at `MAX_SHADOW_DAYS = 28` days, promote if the mean loss difference favours the challenger and $p < 0.10$, otherwise discard it.

The module's own docstring lists why this legacy test is too generous (it pools correlated records as if independent and re-tests daily without adjustment). A corrected version, DM v2, exists behind `TRADER_SHADOW_DM_V2` (default off).

Stage 6: Promotion and hot reload

`shadow.promote_challenger` backs up every champion file to `.prev`, copies the challenger over, and writes the manifest file `{prefix}model_v2.manifest.json` **last**. The bots check `trading_utils.model_reload_key`, which is the manifest's modification time, on every cycle (`_hot_reload_check`). When it changes, they reload. Writing the manifest last means a bot can never load new network weights paired with an old scaler halfway through a copy.

The weekly retrain is a cold restart

Distinguish two hand-offs. The daily promotion above is a genuine hot reload: the bots keep running. The **weekly retrain** is not. `run_pipeline.main` computes the next retrain time with `_next_retrain_time` (defaults `--retrain-day 5`, Saturday, and `--retrain-hour 2`), then calls `_stop_bots` (`SIGTERM`, wait 10 seconds, then kill), runs training on the GPU, and calls `_restart_bots`. The bots read the champion slot at startup. The reason is memory: on an 8 GB device, training and bots cannot comfortably coexist (Chapter 13).

Why flags default OFF, and the runbook

Every change that could alter predictions, labels or trading behaviour ships behind a **flag** whose default reproduces the old behaviour exactly, and a test pins the old code path byte-for-byte (`docs/FLAGS.md`, "Philosophy"). Flags are then turned on in an order set by `research/campaign_2026-`

`08/03_jetson_runbook.md`: Phase 0 baseline reads, Phase 1 run-once instruments, Phase 2 evidence-gated flips (each one only after its instrument's output has been read), Phase 3 one bundled retrain, Phase 4 LLM economics, Phase 5 strategic follow-ons.

Why so much ceremony? Because if you change three things at once and results move, you cannot tell which

change did it; and because any change to the objective or features makes old Optuna scores incomparable. That is "gotcha #2" in CLAUDE.md: after such a change the study databases must be reset. The runbook therefore bundles all such changes into **one** retrain event.

10.4 A worked example: why 20 trades cannot certify anything

Here is the DSR gate computed with the production function `validation.dsr_from_trade_returns`. In every row the model's true per-trade Sharpe ratio is the same, 0.20 (a respectable edge: the average trade earns one fifth of a standard deviation). Only the amount of evidence changes.

Holdout trades	Effective trades	Trials searched	Best Sharpe expected from luck	DSR	Passes 0.60?
20	14	25	0.534	0.13	no
20	14	70	0.642	0.07	no
200	140	25	0.169	0.64	yes
200	140	70	0.203	0.49	no

Read the rows slowly. With only 14 effective trades, the best of 25 *skill-less* trials would be expected to show a per-trade Sharpe of about 0.53 by luck alone, so an honest 0.20 looks like nothing. With ten times the trades, luck's bar falls to 0.17 and the same 0.20 edge passes. Search harder (70 trials) and the bar rises again. The gate is a trade-off between how hard you searched and how much evidence you have. You cannot buy certification by searching more; you can only earn it with more independent trades.

This is not hypothetical. On 2026-09-27 the stock search (25 trials) produced a holdout of 20 trades with 14 effective, Sharpe 0.13 and DSR 0.058, and the model was not saved.

10.5 What the evidence says

- **The pipeline refuses correctly.** In the overnight campaign both books were searched on the clean stores. Crypto: 44 trials, 43 negative, and the winner produced 3 holdout trades, so the gate failed closed on too few trades. Stock: 25 trials, best score +0.024, failed DSR. No model was saved, the meta-label and policy-gate phases ran their "no champion" paths cleanly, and the bots stayed unable to buy. That is the system working as intended.
- **The gate has little power at realistic edges.** SIGNAL's analysis: with $DSR_MIN = 0.60$, a model with a true per-trade Sharpe of 0.10 passes only 1 to 8 % of the time at 10 to 120 effective trades and a pool of 44 trials. The gate protects against luck but will also reject most genuinely modest edges. Remedies (a longer fixed holdout, a holdout-only deflation pool, or relying on shadow evidence) are an owner decision.
- **The policy gate has a structural caveat.** In the default shadow mode, the weekly policy gate replays the *champion*, while the fresh model sits in the challenger slot without a policy replay, unless `GATE_TARGETS_CHALLENGER` (default off) is on. The runbook's Phase 2 flips this after one observed cycle.
- **The Phase-3 bundle is now on.** On the morning of 2026-09-27, on the founder's instruction, `HYPERSEARCH_V3`, `OBJECTIVE_V3` and `TRAINING_REPAIRS_V1` were set to True (with `OBJECTIVE_LONG_ONLY` already True), the old studies were archived, and a 70-trial stock retrain was launched. Its [FLAGS] banner line records exactly which flags were in force. Chapter 14 explains how to read the certificate it produces.

10.6 Further reading

- Marcos López de Prado, *Advances in Financial Machine Learning*, Wiley, 2018. Purging, embargoes, triple-barrier labels, meta-labeling: most of this pipeline's vocabulary.
- David H. Bailey and Marcos López de Prado, "The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting and Non-Normality," *Journal of Portfolio Management* 40(5), 2014.
- David H. Bailey, Jonathan Borwein, Marcos López de Prado and Qiji Jim Zhu, "The Probability of Backtest Overfitting," *Journal of Computational Finance* 20(4), 2017, 39 to 69.
- Francis X. Diebold and Roberto S. Mariano, "Comparing Predictive Accuracy," *Journal of Business and Economic Statistics* 13(3), 1995, 253 to 263; David Harvey, Stephen Leybourne and Paul Newbold, "Testing the Equality of Prediction Mean Squared Errors," *International Journal of Forecasting* 13(2), 1997, 281 to 291.
- R. David McLean and Jeffrey Pontiff, "Does Academic Research Destroy Stock Return Predictability?" *Journal of Finance* 71(1), 2016, 5 to 32.
- In this repository: docs/MAP.md sections 4a to 4d, docs/FLAGS.md, and the runbook linked above.

Chapter 11. Measuring yourself honestly

11.1 The idea

The previous chapter was about testing a model before it trades. This chapter is about testing the *system* after it trades: its gates, its market exposure, and its most expensive advisor, the LLM. The questions are simple to ask and hard to answer honestly:

- Did the gates that blocked trades save money, or did they block winners?
- Is the profit (or loss) skill, or is it just the market going up (or down)?
- Does the LLM's opinion add anything the model did not already know?
- And for each of those: do we have enough data to say?

The last question is the one people skip. The rest of this chapter is largely about it.

Three terms.

- **Alpha** is return that is not explained by exposure to the market. **Beta** is the part that is: a beta of 1.0 to SPY means the book tends to move one for one with the S&P 500. You can buy beta for almost nothing with an index fund. Paying for complexity only makes sense for alpha.
- The **effective sample size** (n_{eff}) is the number of *independent* observations hiding inside a larger number of correlated ones.
- A **verdict** is what a measurement tool concludes: keep, kill, review, or "cannot conclude". A **void** verdict is one that must not be used at all, because its inputs do not describe the system you are asking about.

11.2 Why it matters financially

People are generous graders of their own work. A trader who made 20 % in a year when the market made 25 % may feel skilled; a trader who lost 5 % in a crash may feel unlucky. Both need the same correction: separate what the market did from what the decisions did.

The second trap is counting. Suppose an LLM scores six coins every hour and each score is judged against the next 24 hours of return. In a month that is over four thousand scored rows. But adjacent hours share 23 of their 24 hours of future return, and the six coins mostly move together. Four thousand rows are really closer to thirty independent observations. Statistics computed as if you had four thousand will declare noise to be signal, and a decision to "keep paying for this" or "keep this gate" will be made on nothing.

11.3 How this system does it

The decision report: pricing every veto

`decision_report.py` reads the journals (Chapter 9) and, for every skipped trade, asks what would have happened if the gate had not fired. It replays the vetoed entry through **the same exit kernel** (`policy_exits`) that prices real exits, charges the same round-trip costs, and reports for each gate: how many vetoes, and the mean counterfactual net return. A gate whose vetoed trades would have *lost* 0.8 % on average saved 0.8 % per veto. A gate whose vetoed trades would have *made* money is charging admission, not providing protection.

Three honesty mechanisms are built in:

1. **Episode deduplication.** A symbol skipped every 30 seconds for a whole day would otherwise produce hundreds of overlapping replays of the same price move. Rows are collapsed to one per (symbol, reason, calendar day).
2. **Confidence intervals and a minimum count.** Every mean carries a 90 % bootstrap interval, and no REVIEW / OK / CHANGE verdict fires below `MIN_VERDICT_N = 9`. If the interval includes zero, the answer is "cannot conclude".
3. **Stale and unrepresentative flags.** If prices cannot be fetched or nothing was priced, the report is written with `stale: true` so the GUI and downstream tools refuse to trust it, and a representative flag goes false when fetch failures occurred.

The 2026-09 campaign added a report-only second interval that resamples whole calendar days instead of rows (`ci90_dayclust`), because on a given day all stocks tend to move together; the report now prints how often the two intervals disagree.

The beta ledger: alpha versus beta

`beta_ledger.py` regresses the account's daily equity returns on SPY and BTC. Its docstring cites the key lesson from Asness, Krail and Liew (2001): funds that looked market-neutral had near-zero same-day beta but large **lagged** beta, because their positions were priced with a delay. Summing the same-day and lagged betas roughly doubled the measured exposure and erased much of the apparent alpha. So the ledger reports:

- lagged (summed) betas, not just same-day betas;
- an alpha estimate with a heteroskedasticity-and-autocorrelation-robust (HAC) t-statistic;
- up-market and down-market betas (market timing shows up as an asymmetry);
- betas conditional on the market's trend state;
- since 2026-09, a robust **winsorized** beta after Welch (2022), a `beta_stable` flag, and `alpha_mintrl_years`, the number of years of data needed before an alpha of the observed size could reach a t-statistic of 2.

The LLM scorecard

`llm_eval.py` asks whether the LLM's score `s` adds information *beyond* the model's prediction. This is subtle because the LLM's prompt shows it the model's prediction, so its scores can simply echo the model. The primary test is a regression

```
realized return = a + b1 x (model prediction) + b2 x (standardized LLM score)
```

and the question is whether `b2` is reliably positive. The standard errors are Driscoll-Kraay (1998), clustered by the hour the forecast was made, to cope with overlapping horizons and with all symbols moving together. No verdict is issued unless **all** of these hold (constants at the top of the file): at least `MIN_POWER_N = 60` realized rows, at least `MIN_POWER_T0 = 120` distinct forecast hours, and at least `MIN_EFFECTIVE_N = 20` effective observations, computed as the time span divided by the forecast horizon.

The evidence-reads wrapper

`scripts/evidence_reads.py` runs the whole measurement shelf in one command and prints a table with READY, NOT YET, NO DATA, FAILED or SKIPPED for each read, together with each read's pre-registered sample-size rule and, where history exists, an estimated number of days until it becomes READY.

The anytime-valid ledger

`llm_eprocess.py` is the newest instrument and the most unusual. Ordinary statistical tests assume you decide the sample size in advance and look once. In practice you look every day, and each look is another chance to be fooled. An **e-process** is a running measure of evidence that stays valid no matter how often you look or when you stop.

The intuition is a betting game. Start with \$1 of imaginary capital and, each day, bet a fraction of it that the LLM's size tilt added value that day. If the tilt adds nothing, the game is fair or unfavourable, and a theorem (Ville's inequality) says the chance that your capital ever reaches \$40 is at most 1 in 40. So if it does reach \$40, that is strong evidence, however many times you peeked.

The ledger computes a daily mark-to-market value of the LLM tilt, net of fees and of the LLM bill, in basis points of equity, and runs three one-sided bets: KEEP (the tilt adds value), KILL-HARM (it destroys value) and KILL-FUTILITY (it adds less than 1 basis point per day, about \$10 a day on \$100,000, ten times the spending cap). Any bet whose capital reaches 40 decides. The first decision may come after 10 test days and at least 30 trades; the horizon is 180 days. In simulation the design has an 86.7 % chance to detect a true +1 basis point per day by day 180, but only 38.7 % by day 90. The module **refuses to run on live data** (exit code 3) until the owner signs its pre-registration sheet, `research/campaign_2026-09_jetson/llm_eprocess_params.json`, because thresholds chosen after seeing the data are not evidence.

Effective sample sizes, and a rule against double counting

Overlap is handled in several places: `sample_weights.average_uniqueness` and `calendar_effective_n` for trades that overlap in time, the span-over-horizon rule in `llm_eval`, and day-clustering in `decision_report`. CLAUDE.md gotcha #4 states the rule: use **one** correction for overlap. The uniqueness-based `n_eff` and the Lo (2002) serial-correlation factor are two estimates of the same inflation; applying both punishes the data twice.

What "void" means

A verdict is void when the data behind it came from a different system than the one you are judging. The clearest case in this repository: the April and May 2026 journals were written while a since-removed rule blocked buys whenever the LLM score was below 0.60. Of 11,818 skip rows in those journals, 11,815 were that one gate. Today's code only vetoes below 0.15. Any gate attribution, LLM verdict or spend ledger computed from those journals describes a policy that no longer exists, so the runbook declares pre-campaign `decision_report` figures void and the LLM ledger excludes journals through 2026-05-07. Void is stronger than "not significant": a not-significant result is weak evidence about the current system, while a void result is no evidence about it at all.

11.4 A worked example: 720 rows or 30?

Suppose one month of hourly LLM scores for one coin: 30 days x 24 hours = 720 scored rows, each judged against the next 24 hours. You find a correlation of 0.10 between score and outcome.

The usual t-statistic for a correlation is $r \times \sqrt{(n - 2) / (1 - r^2)}$.

- Treating the rows as independent, $n = 720$: $t = 0.10 \times 26.8 / 0.995 = \mathbf{2.69}$. That clears the conventional bar of 2 and looks like a finding.
- But each 24-hour outcome overlaps the next 23. The effective count is the span divided by the horizon, $720 / 24 = 30$. With $n = 30$: $t = 0.10 \times 5.29 / 0.995 = \mathbf{0.53}$. That is nothing.

Same data, same correlation, opposite conclusions. The `MIN_EFFECTIVE_N = 20` floor in `llm_eval` exists to

prevent the first reading. It also explains why "we have lots of rows" is never a reason to trust an LLM verdict after a few days.

A second example, from the live beta ledger run during the campaign (INTEL owner item 10). The book's summed lagged BTC beta was **+1.18**, while the same-day beta was +0.25 and a plain regression gave +0.44. Looking only at same-day numbers, you would have concluded the crypto book carried a quarter of Bitcoin's risk; it carried more than all of it. The same run printed `alpha_mintr1_years` of 2.0 against a 0.4 year window: the alpha is not estimable yet, and "beta only" is the honest summary.

11.5 What the evidence says

- **The LLM keep/kill test was too eager.** The INTEL department simulated the scorecard's test on synthetic data with no true effect, shaped like this system's data (six symbols, 24-hour horizon). At a nominal 5 % false-alarm rate, the Driscoll-Kraay test with its $t(G-1)$ reference rejected a true null 13.0 %, 11.0 % and 9.6 % of the time at 20, 30 and 60 effective observations. The Ibragimov-Müller alternative (split the sample into 8 blocks, estimate in each, and t-test the 8 estimates) gave 5.6 %, 6.2 % and 5.2 %, the only estimator inside the pre-registered 3 to 7 % band. Both p-values are now printed side by side; making Ibragimov-Müller the deciding test is an open owner decision (INTEL owner items [\[research/campaign_2026-09_jetson/session_artifacts/generals/intel/OWNER_ITEMS.md\]](#), item 1).
- **There is no LLM verdict yet.** The scorecard stands at 0 of 120 required forecast-hour clusters on journals written by the current code. At one book scored every 10 minutes, reaching the floor takes weeks of normal trading, and the clock only starts once a certified model trades.
- **Alpha cannot be established on a paper-trading horizon.** An earlier INTEL estimate put the time needed for a t-statistic of 2 at about 4.3 years, even at a Sharpe near 1. Beta, by contrast, is measurable within months. The recommended policy is to report beta and state that alpha is not estimable, rather than to report a noisy alpha.
- **The anytime-valid ledger has a known weakness.** With realistic day-to-day autocorrelation in the daily statistic (0.3), the simulated false-fire rate rose to 3.5 to 5.4 % against a 2.5 % target. The INTEL department asked the owner to rule on a guard before signing the sheet.

11.6 Further reading

- Andrew W. Lo, "The Statistics of Sharpe Ratios," *Financial Analysts Journal* 58(4), 2002, 36 to 52. How uncertain a Sharpe ratio really is, and why serial correlation matters.
- Clifford S. Asness, Robert Krail and John M. Liew, "Do Hedge Funds Hedge?" *Journal of Portfolio Management* 28(1), 2001, 6 to 19. Lagged betas.
- John C. Driscoll and Aart C. Kraay, "Consistent Covariance Matrix Estimation with Spatially Dependent Panel Data," *Review of Economics and Statistics* 80(4), 1998, 549 to 560.
- Rustam Ibragimov and Ulrich K. Müller, "t-Statistic Based Correlation and Heterogeneity Robust Inference," *Journal of Business and Economic Statistics* 28(4), 2010, 453 to 468.
- Aaditya Ramdas, Peter Grünwald, Vladimir Vovk and Glenn Shafer, "Game-Theoretic Statistics and Safe Anytime-Valid Inference," *Statistical Science* 38(4), 2023, 576 to 601.
- Ian Waudby-Smith and Aaditya Ramdas, "Estimating Means of Bounded Random Variables by Betting," *Journal of the Royal Statistical Society Series B* 86(1), 2024, 1 to 27.
- In this repository: `docs/MAP.md` section 4g (what each instrument is for) and `research/campaign_2026-09_jetson/research_intel.md` (Scouts B and C).

Chapter 12. The LLM as a bounded advisor

12.1 The idea

A large language model can read a headline, an earnings summary and a macro calendar and form a qualitative opinion in seconds. The numerical model in this system cannot read at all; it sees only prices and engineered features. So there is an obvious temptation: let the LLM decide.

This system resists that temptation on purpose. The LLM is an **advisor with a leash**. It can shrink or modestly grow a position, it can veto an entry in a clear catastrophe, and that is all. It never chooses what to buy. Its answers must arrive in a fixed machine-readable shape. If it fails, the trade goes ahead as if it had said nothing. And it has a hard daily budget of one dollar.

This chapter explains each part of that leash, what it cost to run last night, and why the evidence does not yet justify loosening it.

Terms used below:

- A **role** is a job the LLM does: analyst (scores trade candidates), sentiment (scores news for the sentiment feature and gate) and backfill (re-scores historical articles in bulk).
- **Schema-enforced output** means the provider is told the exact JSON shape required, and its API refuses to return anything else.
- **Fail-open** means that when the component fails, the system behaves as if the component were not there. Its opposite, **fail-closed**, means that on failure the system does nothing.

12.2 Why it matters financially

Three reasons, each of which costs money if ignored.

1. **LLMs remember the past they are asked to predict.** A model trained on text up to some date has, in a sense, read the future of every headline before that date. Glasserman and Lin (2023) showed that LLM sentiment scores on in-sample headlines carry look-ahead bias, and 2026 work collected in the INTEL research log found that effective knowledge cutoffs can sit months before the vendor's stated date. Any backtest of LLM opinions on historical news is therefore suspect. Only live, post-cutoff decisions can judge the advisor.
2. **LLMs are persuadable.** The prompt in this system shows the model's own prediction. Research summarised in the same log found that context changes how an LLM interprets identical evidence, so the LLM may simply echo the model. An echo adds cost and no information.
3. **API calls are a running cost with no ceiling unless you build one.** A call every ten minutes per book is 288 calls a day for two books. At a few tenths of a cent each that is trivial; at a cent each it is a few dollars a day, every day, whether or not the calls help.

The literature on real LLM trading skill is thin and mixed. Lopez-Lira and Tang found that GPT-4 headline scores capture the immediate (non-tradable) market reaction well, and that the tradable drift after it is concentrated in small stocks and negative news, which is outside this book's liquid long-only universe. An audit of 77 LLM trading studies, cited in the INTEL log, found that of the 19 closed-loop studies at its core only two used time-consistent splits and only one modelled transaction costs.

12.3 How this system does it

Where the LLM sits in the loop

In the cycle described in Chapter 9, `_run_llm_analysis` runs at most every `LLM_INTERVAL_SEC = 600` seconds per book. It sends all current candidates in one call to `llm_analyst.analyze_trades(llm_analyst.py)`, which returns a score `s` between 0 and 1 for each symbol plus short bull, bear and summary strings. The system prompt tells the LLM exactly how its number will be used:

- `s < 0.15` (`LLM_VETO_THRESHOLD` in `trading_utils.py`): a **veto**. New buys in that symbol are blocked immediately. If two consecutive analyses both veto, an open position is **liquidated** (the "veto strike" counter in `base_loop.py`). The prompt reserves this for confirmed catastrophe: fraud, insolvency, delisting, a hack.
- Otherwise the size multiplier is `llm_mult = 0.5 + s`, so between 0.65 and 1.5, which enters the clamped tilt product in `_compute_position_size`. A score of 0.5 changes nothing.

The prompt also contains a security instruction: headlines are untrusted data scraped from the internet and may contain planted instructions; the LLM must judge the news, not obey it. The analyst runs at temperature 0.2 (`_ANALYST_TEMPERATURE`) with a 45 second timeout.

Schema-enforced outputs, three ways

`llm_client.py` speaks to three providers without any vendor SDK, and forces each one to return valid JSON matching the schema built in `llm_analyst.py` (required fields `s`, `bull`, `bear`, `r`):

- **Gemini**: `responseMimeType` plus `responseSchema`.
- **Anthropic (Claude)**: forced tool use. The schema becomes a tool's input schema and `tool_choice` pins the model to that tool, so its only possible answer is a filled-in form.
- **OpenAI** and OpenAI-compatible endpoints (OpenRouter, Groq, a local Ollama): `response_format` with a strict JSON schema.

The practical benefit is that the caller never parses free prose, and a malformed answer is a detectable failure rather than a silently misread number.

Which model answers: roles, chains and selection modes

`resolve_provider_chain(role, config)` returns an ordered list of (provider, model) candidates, and `call_llm` tries them in order, skipping providers that are rate-limited, over budget or missing a key. The `selection_mode` in `llm_config.json` controls the order:

- `auto` (the default): Anthropic, then OpenAI, then Gemini, each skipped if no key is configured, with every enabled extra endpoint appended last;
- `single`: only the configured provider, no fallback;
- `free-only` and `best-free`: only free endpoints, with Gemini's free tier as the last resort.

The `backfill` role is pinned to Gemini because it uses Gemini's discounted Batch API. On this Jetson only a Gemini key is configured, so in practice every call goes to Gemini.

Fail-open, everywhere

The module docstring of `llm_analyst.py` states the contract: "On any failure, returns {} for pass-through (never blocks trades)." Concretely:

- a transport error, timeout, refusal or unparseable answer returns an empty result;
- the loop then keeps the previous scores (`llm_scores` are deliberately untouched), and a symbol with no score is treated as `s = 0.5`, which means no veto and a multiplier of exactly 1.0;
- scores older than `LLM_SCORE_TTL_SEC = 7200` seconds expire, again to neutral;
- consecutive failures back off exponentially, 600 s, 1,200 s, 2,400 s, capped at 3,600 s, and each is journaled as an `llm_backoff` row.

Contrast this with the rest of the live path, which is fail-closed: a missing prediction or quote blocks the entry. The asymmetry is deliberate. The model is the signal, so without it you must not trade; the LLM is an overlay, so without it you trade as if it were neutral.

The one-dollar cap

`_DAILY_COST_LIMIT = 1.00` (in `llm_client.py`). Every call's cost is computed from the provider's reported token usage and a pricing table (for example Gemini 2.5 Flash at \$0.30 per million input tokens and \$2.50 per million output tokens; Flash-Lite at \$0.10 and \$0.40). The running total is shared across processes through `llm_cost.json`. Once the day's spend reaches the cap, every further call is refused, and fail-open turns that refusal into neutral scores. The day resets at midnight Pacific time (`_maybe_reset_quota`), matching Google's quota reset. Note that the cap is global: an expensive analyst can starve the sentiment role of budget.

The free-first plan

The 2026-08 campaign built the plumbing to run the LLM at zero cost on free endpoints, gated by a qualification script, `scripts/llm_qualify.py`, which checks a candidate's reliability and its agreement with the production analyst before any switch. The runbook's Phase 4 then says: read the spend verdict first; only if it says "keep" and a free model qualifies, add budget rows for it and switch `selection_mode`. As of 2026-09-27 this plan is **not viable as the default**: no Groq or OpenRouter keys exist, no local model runs, and the arithmetic does not work. Groq's free tier allows about 200,000 tokens a day, which is roughly 40 analyst calls of 5,000 tokens against a need of up to 288. OpenRouter's free models allow 50 requests a day, or 1,000 after a one-time purchase of at least \$10 of credit. A local LLM on the Jetson's CPU would need several minutes per analyst prompt, far too slow for a 10-minute cadence.

12.4 A worked example: last night's bill

The journal for 2026-09-27 records every analyst call made while the bots ran overnight (crypto book only, 03:10 to 08:58 Central time). Summing the `cost_usd` field of the `llm_analysis` rows:

Model chosen by the router	Calls	Total cost	Cost per call
gemini-2.5-flash-lite	23	\$0.0244	\$0.0011
gemini-2.5-flash	11	\$0.1067	\$0.0097
Total	34	\$0.1311	\$0.0039

Median latency was 5.3 seconds; the slowest call took 30.0 seconds, which is the configured `max_llm_latency_sec` ceiling. One call failed and produced an `llm_backoff` row.

Three lessons come out of this small table.

1. **The model mix is the budget.** Flash cost nine times Flash-Lite per call. It answered a third of the calls and ran up 81 % of the bill.

2. **Extrapolate before you scale.** Six calls an hour is 144 calls a day per book. At last night's average of \$0.0039, that is about \$0.56 a day for one book, and about \$1.12 for two, which is over the \$1.00 cap. At Flash-Lite prices throughout, one book would cost about \$0.15 a day. The cap would have started refusing calls late in the day with both books running.
3. **Spend is not value.** Every score in those rows has "pred": null: no model was loaded, and entries were impossible (the bots were in an exits-only mode). The LLM still scored every candidate every ten minutes, because the halt blocks entries but does not silence the analyst, and a two-strike veto could still have sold a position. The ENGINE department raised this as an owner question (item 20). Thirteen cents bought no decision-relevant information.

A second, smaller example shows the leash in action on a single order. The stock book is about to buy \$5,000 of a stock. The LLM scored it 0.62, so `llm_mult = 1.12`. That factor joins the other tilts, and the product is clamped between 0.1 and 1.30, so even a perfect score cannot push the order beyond 1.3 times its base. Had the score been 0.12, the buy would have been skipped and journaled as an `llm_veto`; if the next analysis ten minutes later also said below 0.15, any open position in that name would have been sold.

12.5 What the evidence says

- **There is no verdict on whether the LLM earns its keep.** The scorecard (`llm_eval.py`, Chapter 11) needs 120 distinct forecast hours and 20 effective observations on journals written by the current code, and it has none. Older journals are void because they ran a different gate.
- **Two cheap experiments are designed but not run.** A test-retest check (ask the same question twice and measure how much `s` moves; if vetoes flip on repeat calls they are partly coin tosses) and a pred-blind A/B (hide the model's prediction from half the prompts to measure echo) would together cost about \$0.75. They are parked as a money ask for the founder.
- **The analyst is shown the model's prediction by default** (`include_pred=True`), which is the anchoring risk described above. Whether to hide it is a decision to make after the A/B.
- **Local models are not the answer on this hardware.** Measurements on the Jetson put a FinBERT-sized encoder at about 134 ms per headline, which is feasible, but 2026 studies found next-day information coefficients of about 0.014 at best for such scorers, not enough to justify a new dependency.
- **Reliability is now measurable.** Since the campaign, every analyst call, including failures, writes an `llm_call` row with provider, model, latency and failure reason, so the next weeks of operation will show how often the fail-open path is actually taken.

The honest summary: the LLM is cheap, bounded and unproven. The leash is the right design until a live, pre-registered measurement says otherwise.

12.6 Further reading

- Alejandro Lopez-Lira and Yuehua Tang, "Can ChatGPT Forecast Stock Price Movements? Return Predictability and Large Language Models," arXiv:2304.07619 (revised through 2025).
- Paul Glasserman and Caden Lin, "Assessing Look-Ahead Bias in Stock Return Predictions Generated by GPT Sentiment Analysis," arXiv:2309.17322, 2023.
- Provider documentation on structured outputs: Google's Gemini API documentation on structured output and pricing (ai.google.dev/gemini-api/docs), Anthropic's documentation on tool use (docs.anthropic.com), and OpenAI's documentation on structured outputs (platform.openai.com/docs).
- In this repository: `research/campaign_2026-09_jetson/research_intel.md` (Scout A, with about thirty dated 2026 sources), the runbook's Phase 4 in `03_jetson_runbook.md`, and `docs/MAP.md` section 9

items 12 and 13.

Chapter 13. Running on an 8 GB Jetson

13.1 The idea

This whole system, data harvesting, neural-network training, two trading loops, a GUI and a supervisor, runs on one NVIDIA Jetson Orin Nano: a credit-card-sized computer with six ARM cores, a small GPU and **8 GB of memory shared between the CPU and the GPU**. There is no cloud server behind it. When the Jetson runs out of memory, overheats, fills its disk or loses power, the trading stops, or worse, half-stops.

This chapter is about operating a trading system as a piece of machinery: how memory is budgeted, which processes may use the GPU, how the service is supervised and restarted, what happens when memory runs out, how the logs are kept from filling the disk, and how the owner is told when something breaks. Operations is not glamorous, but an unattended trading system is only as good as its worst night.

Some vocabulary.

- **RSS** (resident set size) is the physical memory a process is using right now.
- **Swap** is disk space used as overflow memory. It prevents crashes but is far slower.
- The **OOM killer** ("out of memory") is the Linux kernel's last resort: when memory is exhausted it kills a process to save the machine.
- **systemd** is the Linux service manager. A **unit** is its description of a service: what to run, how to restart it, what limits apply.
- A **watchdog** is a timer the service must keep resetting; if it stops, systemd assumes the service is hung and restarts it.

13.2 Why it matters financially

Operational failures cost money in ways that never appear in a backtest.

- A bot that crashes while holding positions leaves them managed only by whatever stop orders are resting at the broker. Software-side trailing stops, signal exits and the end-of-day flatten all stop.
- A bot that crash-loops can do damage by restarting: each restart cancels and re-replaces orders, and each gap between cancel and re-place is a moment without protection.
- An alert that never arrives turns a five-minute problem into a five-week one. This system's bots were down from 2026-05-07 to 2026-09-26 and nothing told anyone.
- A training run that starves the bots of memory can make them miss exits during exactly the hours the retrain runs.

Professional trading firms treat these as risk controls, not IT chores. The Futures Industry Association's guidance on automated trading, cited in the INTEL research log, describes a kill switch that blocks new orders *and* cancels working ones, and warns the operator of the consequences. That standard is useful to measure this system against.

13.3 How this system does it

The memory budget

The machine reports 7,619 MB of total memory and a 12 GB swapfile on the NVMe drive

(scripts/setup_jetson_system.sh replaces the default compressed-RAM swap with this swapfile, because compressed swap costs CPU and memory). The big consumers, as measured by the ENGINE department on this box:

- **PyTorch itself.** Importing torch adds about 331 MB of resident memory to a process. The trading model is tiny by comparison: 293,761 parameters, about 1.2 MB. Quantizing the weights therefore saves almost nothing; the runtime is the cost.
- **A CUDA context.** Each process that touches the GPU reserves roughly 300 MB for it (the comment above BOT_ENV in run_pipeline.py).
- **Training data.** The trainer keeps one scaled feature matrix per fold in memory at a time (ScaledCache); the current stock run logs about 50 MB per matrix.

The design responses follow directly:

1. **Bots never touch the GPU.** run_pipeline.BOT_ENV sets CUDA_VISIBLE_DEVICES='', OMP_NUM_THREADS=2 and TORCH_NUM_THREADS=2 for every bot process, and run_bots.py itself defaults CUDA_VISIBLE_DEVICES to empty. Inference of a small LSTM on two CPU threads takes tens of milliseconds, well inside a 30 second cycle, and saves the CUDA context and GPU memory for training.
2. **Both books in one process.** Combined mode (--combined-bots) runs both loops as threads in one Python process, so PyTorch and the libraries are loaded once instead of twice.
3. **Training holds a lock.** gpu_lock.py provides an exclusive file lock (fcntl.flock on .gpu.lock). The trainer must acquire it before using the GPU, so two training runs can never share the 8 GB. The operating system releases the lock automatically if the holder dies, so there are no stale locks to clean up.
4. **The weekly retrain stops the bots.** _stop_bots sends each bot SIGTERM, waits 10 seconds, then kills it, and sleeps 3 seconds so the kernel can reclaim the memory before training starts. Its docstring states the reason: each bot's PyTorch import reserves 300 to 500 MB, and stopping them frees about a gigabyte for CUDA. The cost is that the books are unmanaged by software during the retrain (Chapter 10).
5. **A measured alternative is parked.** The ENGINE department built a pure-numpy replica of the LSTM's forward pass. On 3,207 real feature windows it changed zero trading decisions and would cut about 349 MB from each bot process by never importing PyTorch. It is not wired in, because its pre-registered accuracy bar (1e-6) turned out to be stricter than PyTorch's own float32 noise; re-registering that bar is an owner decision (ENGINE owner item 14).

Heat

Two thermal rules exist. Before any training phase, run_pipeline._bounded_thermal_wait waits for the GPU to cool below 70 °C, for at most 30 minutes. In the bots, _sleep doubles the cycle interval when the GPU is above THERMAL_THROTTLE_TEMP = 75 °C. The temperature comes from the Linux thermal sysfs files via hw_monitor.py get_gpu_temp.

The service

Since 09:22 on 2026-09-27 the system runs as a **systemd user unit**, which needs no administrator password. Its key lines (from ~/.config/systemd/user/trader.service):

```
Type=notify
ExecStart=/home/kyle/miniforge3/envs/jetson/bin/python -u run_pipeline.py --combined-bots --bot-only
EnvironmentFile=-/home/kyle/trader/.env
Restart=on-failure
RestartSec=30
```

```
WatchdogSec=900
OOMPolicy=continue
OOMScoreAdjust=200
MemoryMax=6G
```

A drop-in override in `trader.service.d/override.conf` replaces `ExecStart` to add `--stock-only`, recording the founder's decision without editing the repository's generic unit. **Linger** is enabled (`logind enable-linger kyle`), so the user's service manager starts at boot and keeps running after logout.

What each line buys:

- `Type=notify` and `WatchdogSec=900`: `run_pipeline` sends `READY=1` when it is up, and a heartbeat thread sends `WATCHDOG=1`, but **only if the main loop has stamped progress within the last 600 seconds** (`WATCHDOG_STALL_SEC`). Before this rule, the heartbeat kept the watchdog happy even when the main thread was stuck forever on a blocked read, so the watchdog only caught outright death. Now a real hang lets the lease expire and systemd restarts the service. A thermal wait counts as progress, so cooling down is not mistaken for hanging.
- `Restart=on-failure` with `RestartSec=30`: a crashed pipeline comes back in 30 seconds.
- `MemoryMax=6G`: the whole unit, bots and any training it launches, may not exceed 6 GB, which leaves room for the operating system and the GUI.
- `OOMScoreAdjust=200`: if the kernel must kill something, it should prefer the pipeline to a random system process.
- `OOMPolicy=continue`: when the OOM killer kills one *child* (say, a training phase), systemd does not tear down the entire unit; the pipeline's own retry and bot-restart logic handles it.

The internal supervisor and the crash loop

Inside the unit, `run_pipeline` checks its bots every 60 seconds and restarts any that exited (`_check_restart_bots`). The historical problem: it restarted forever at a fixed 60 seconds, with no backoff and no notion that the same crash was repeating. In April 2026 one deterministic bug (a string formatted as a number in `fundamentals.py`, since fixed) crashed the stock bot **1,778 times**. A backoff design is now in the tree behind `TRADER_BOT_RESTART_BACKOFF` (default off): repeated identical crashes wait 60, 120, 240 seconds and so on up to 960, and after five identical crashes within an hour the supervisor stops restarting that bot and sends one critical alert. Replayed against the real April history it cut 1,616 restarts to 12. Since June 2026, errors inside a cycle are caught in-process instead of crashing the bot, which changes the failure mode: the April bug would now repeat every cycle with a deduplicated warning instead of restarting.

Logs and journals

Three kinds of files grow.

- **logs/trader.log**, the structured log from `log_config.py`: rotating, 10 MB per file, five backups (`_MAX_BYTES`, `_BACKUP_COUNT`), with a lock so several processes can share it safely. Tests used to write into this production log (33,452 lines in one night were test noise), which made forensics impossible; the campaign added `TRADER_LOG_DIR` so tests log elsewhere.
- **Bot stdout logs** (`crypto_bot_output.log`, `stock_bot_output.log`; combined mode writes to the crypto one). These are rotated one-deep at 20 MB, but only when a bot is launched (`_rotate_log`). A bot that runs for months never rotates. The file `stock_bot_output.log` on this box is 115 MB, last written on 2026-05-07, left over from the crash loop.
- **Journals** (`journals/`): 5.5 MB across 53 entries today. Compression of old journals exists

(TRADER_JOURNAL_ROTATE_DAYS), but is off by default because several readers open the plain files directly and would go blind to compressed days.

The authoritative list of every generated file, who writes it and who reads it is docs/STATE_FILES.md.

Alerts

notify.py sends alerts to a webhook (TRADER_WEBHOOK_URL, Discord- or Slack-compatible) or Telegram (TRADER_TELEGRAM_BOT_TOKEN plus TRADER_TELEGRAM_CHAT_ID). It never blocks a trading path (sends happen on a background thread), deduplicates each alert key to once per 10 minutes, and retries a failed critical send once. With no channel configured it does nothing, silently.

13.4 A worked example: the memory ledger during a retrain

At 09:57 on 2026-09-27, with the Phase-3 stock retrain running and the stock-only service up, the machine looked like this (free -m and /proc/<pid>/smaps_rollup):

Item	Memory
Total physical memory	7,619 MB
Used	5,086 MB
Available (free plus reclaimable cache)	2,296 MB
Swap in use	2,976 MB of 12,001 MB
hypersearch_v2.py (training, on the GPU)	1,357 MB RSS
run_bots.py --stock-only	95 MB RSS
run_pipeline.py (the supervisor)	28 MB RSS

How to read it. First, the trainer is the elephant: over a gigabyte, and on this unified-memory machine its GPU allocations come out of the same 7.6 GB. Second, the bot looks small at 95 MB, but that is because the market is closed on a Sunday and much of the idle process has been pushed to swap; the ENGINE measurement of a live bot with PyTorch loaded is several hundred megabytes. When the market opens and the bot touches those pages again, they must come back from disk, which is why swap is a safety net and not a budget. Third, nearly 3 GB of swap in use means the machine has been under pressure at some point during the night; "available" is the number to watch, and 2.3 GB is comfortable but not generous. Finally, this particular retrain was launched by hand for the campaign, outside the service, so it is not counted against the unit's MemoryMax=6G. A retrain launched by run_pipeline itself would be.

13.5 What the evidence says

- **The most valuable operational fix costs one minute.** The .env on this box contains only the Alpaca and Finnhub keys; no webhook or Telegram variables. Every breaker trip, crash and halt alert has therefore been silent. The ENGINE department estimated 232 alerts would have fired during April's crash loop; none was sent (ENGINE owner item 17).
- **The user unit is installed but not yet proven by a reboot.** The pre-registered acceptance test (reboot without logging in and see the service come up within 180 seconds; kill -9 the main process and see it

return within 60; freeze it with `kill -STOP` and see the watchdog restart it) is written in `research_engine.md` as experiment X8. Until it is run, treat automatic recovery as expected, not demonstrated. Note too that this box runs `systemd 249`, which lacks the newer escalating restart-delay options, so backoff has to live in the pipeline.

- **Halt is entries-only.** Creating `trading_halt.flag` blocks new entries but does not cancel working orders, and the LLM keeps running. Whether a halt should also cancel working buys is staged as `HALT_CANCELS_WORKING_BUYS` (default off) and awaits the owner.
- **Crash-loop backoff is designed and replay-tested but off.** The numbers (5 crashes, 60 minutes, 960 seconds) are judgment calls, and the choice between giving up permanently and probing again after an hour is an owner decision.

13.6 Further reading

- Betsy Beyer, Chris Jones, Jennifer Petoff and Niall Richard Murphy (eds.), *Site Reliability Engineering: How Google Runs Production Systems*, O'Reilly, 2016. Monitoring, alerting and post-mortems for systems that must not stop.
- Michael T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed., Pragmatic Bookshelf, 2018. Circuit breakers, timeouts, bulkheads and the "steady state" pattern, which is exactly the log and journal problem above.
- The `systemd` manual pages for `systemd.service` and `systemd.resource-control` (freedesktop.org/software/systemd/man), for `WatchdogSec`, `OOMPPolicy` and `MemoryMax`; and `loginctl(1)` for `linger`.
- In this repository: `docs/MAP.md` section 4i, `docs/STATE_FILES.md` section 10 (logs), and the `ENGINE` research log sections E5 and E6.

Chapter 14. What the evidence says today, and what would change it

14.1 The idea

Every earlier chapter described machinery. This one reports the result of running it. On the night of 2026-09-26 to 27, for the first time, the current code was run end to end on the production Jetson, against clean data, with every guard switched on. The honest headline is short:

On clean, leak-free data, the current model and objective showed no measurable ability to pick winners in either book.

That is not a failure of the pipeline. It is the pipeline doing its job: refusing to certify something that has not been shown to work. This chapter explains how that conclusion was reached, why it is more useful than it sounds, what levers could change it, what the retrain running as this book is written is testing, how to read the certificate it will produce, and why the crypto book was set aside.

Terms:

- **Selection skill** means the model's ranking of opportunities is better than a random ranking. Without it, trading is paying costs to hold a random portfolio.
- A **zero-skill ranker** is a benchmark: a random choice of trades with the same frequency and the same costs as the model's. If the model cannot beat it, the model has no skill at that task.
- A **lever** is a design change that could plausibly create or reveal skill, as opposed to simply searching the same design longer.

14.2 Why it matters financially

Most people who build trading systems never get a clean negative result. They get a noisy positive one, trade it, and learn the truth from their account statement. A clean negative is valuable for three reasons.

1. **It is cheap.** It cost a night of GPU time and no capital.
2. **It is specific.** The diagnosis below names *why* the scores were negative, which points at what to change.
3. **It sets the prior.** The repository's own map (docs/MAP.md section 9, item 1) had already estimated, from a 2026-07 review, that even a model passing every gate had perhaps a 40 to 55 % chance of being genuinely profitable, and that live Sharpe ratios typically come in 50 to 73 % below backtest. Knowing that before money is at stake is the point.

14.3 How the result was reached

The two searches

Both stores were rebuilt from scratch (Chapter 10), and both books were searched with the long-only objective.

- **Crypto:** 44 trials in the study; 43 scored negative and one scored +0.03. The winner's holdout produced **3 trades** in 31,662 rows, so the gate refused to save it (`insufficient_n`).

- **Stock:** 25 trials; the best score was +0.024. Its holdout had 20 trades (14 effective), a Sharpe of 0.13 and a DSR of 0.058 against the 0.60 bar, so it was not saved.

A search that finds nothing proves little by itself: maybe it needed more trials. So the SIGNAL department decomposed the scores, trial by trial and fold by fold, using attributes the trainer now records (gross return, cost drag, trade count, hit rate, threshold pass rate per fold), and pre-registered the classification rules before looking.

What the decomposition found

- **Crypto (DECOMP-1, 16 trials):** 44 of 48 fold Sharpes were below zero. Every searched trade threshold sat below the live admission floor of about 1.2 %, the minimum predicted return the live cost gate demands, and half sat below the 0.60 % round-trip cost itself. In plain words: the search kept choosing rules that trade on forecasts too small to pay for the trade. More trials of the same design would not fix that.
- **Stock (DECOMP-2, first 11 trials):** no trial beat the zero-skill ranker by more than one standard error. The random ranker's expected Sharpe even predicted the *ordering* of the trials (rank correlation 0.65, $p = 0.03$; for crypto 0.51): trials scored well or badly for reasons a random ranker shares, such as how many trades they happened to take, not because of what they predicted. The first fold was zero or negative in all 11 trials. Unlike crypto, 9 of 11 stock thresholds cleared the live floor, and the raw drift of the target exceeded costs; the sign was lost through the scoring formula's $-0.5 \times \text{std}$ term and a regime penalty.
- **The study's "best" stock trial was a trade that never happened.** Its score was exactly 0.0, produced by `compute_sharpe`'s rule of returning 0.0 when a fold has fewer than 10 trades (it took 0, 0 and 6). In a study where everything else is negative, zero wins. The fix, `MIN_TRADE_PRUNE`, is staged and proven in the landing queue but not yet in the tree.
- **A penalty with the wrong sign.** The objective multiplies a trial's score by 0.7 when its worst regime is bad. For a positive score that is a penalty. For a negative score, multiplying by 0.7 moves it *toward* zero, which is a reward. It fired on 15 of 15 negative crypto trials and 6 of 11 negative stock trials. `TRAINING_REPAIRS_V1` makes it sign-correct.

The cost side

Separately, the ENGINE department reconstructed 686 filled crypto paper orders from February to April 2026. The realized round-trip cost of crossing the spread on LINK, XRP and DOGE was about 30, 26 and 34 basis points, against the 10 basis points (`FLAT_SPREAD_PCT['crypto'] = 0.10` in `fees.py`) the backtester, the meta-labeler and the cost gate assume. Paper trading has no queue or market impact, so these are floors on real cost. A model trained and certified on a cost three times too low will admit trades the venue does not pay for.

14.4 The levers

Each lever below is staged behind a default-off flag or written up as an owner proposal. None is a promise; each is a hypothesis with a test.

Lever	What it changes	Status
Minimum-trade prune	a trial with too few trades is pruned instead of scoring 0.0	staged (<code>MIN_TRADE_PRUNE</code>), not landed
Cost-anchored thresholds	the searched threshold range starts at the cost	in <code>OBJECTIVE_V3</code> , now on
Live-floor threshold	the trainer applies the same admission floor as	proposal

Lever	What it changes	Status
	the live gate	(OBJECTIVE_THRESHOLD_FLOOR_LIVE)
Drift- or benchmark-relative score	judge trades against the market's drift, not zero	proposal
Penalty form	fix the sign of the regime penalty, reconsider - $0.5 \times \text{std}$	sign fix in TRAINING_REPAIRS_V1, now on
Session mask	score stock entries only inside the live entry windows	OBJECTIVE_SESSION_MASK, off
Startup count	60 random trials before the sampler steers; a 25 or 40 trial run never steers	proposal (startup_trials_owner_item.md)
Honest crypto costs	spread measured, not assumed	census tool built; ruling pending

14.5 The Phase-3 retrain

On the morning of 2026-09-27, on the founder's explicit instruction, the Phase-3 bundle from the runbook was switched on: HYPERSEARCH_V3 (final refit, the LightGBM leg before the gate, a fitted blend weight, certification of the blended model), OBJECTIVE_V3 (per-ticker resets, cost-anchored threshold range) and TRAINING_REPAIRS_V1 (validation loss on the trial's own criterion, the regime-penalty sign fix, a look-ahead repair and bar-based embargoes), with OBJECTIVE_LONG_ONLY already on. The old studies were archived, and a 70-trial stock search began at 09:21.

At 10:07 eight trials had reported. Six scored between -0.39 and +0.15. One (trial 3) was pruned after a CUDA memory allocator error on the Jetson; because FAILED_TRIAL_PRUNE was active for this run, the failure was recorded as pruned rather than as a fake 0.0 score, which is exactly the protection the previous section called for. At about five minutes per trial the search will run for several hours. Remember that with PRUNE_STARTUP_TRIALS = 60, the first 60 trials are random, so the sampler will steer for only the last 10.

Do not read the in-search scores as results. A +0.15 in-search score is exactly the kind of number selection bias produces. Only the holdout certificate counts.

14.6 How to read the next certificate

When the search ends, the trainer prints either Model NOT saved with the holdout report, or New best model with the certificate. The certificate lives in stock_config_v2.pkl under holdout. Read it in this order.

1. **n_trades** and **n_eff**. How many holdout trades, and how many effectively independent ones. Below 10 effective trades nothing else matters. Below about 100, expect wide uncertainty.
2. **sharpe**. Must be positive. Note whether it is per trade or annualized in the printout.
3. **n_trials_pool**. How many configurations the winner was chosen from. This sets the luck bar.
4. **dsr** against $\text{DSR_MIN} = 0.60$. The probability that the Sharpe beats the best of that many skill-less trials.
5. **min_trl**. The minimum track record length: how many effective trades would be needed for this Sharpe

to clear the bar. If it is far above `n_eff`, the certificate is thin.

6. **hit_rate and pred_deciles.** Do the model's highest predictions actually earn the most? A real ranker shows outcomes rising across prediction deciles, not a single lucky decile.
7. For a blended certificate: **lstm_weight** (how much the neural net contributes versus LightGBM), **q10_vetoed**, and **cs_rank_ic** for each leg (the cross-sectional rank correlation between predictions and outcomes; positive and stable is what skill looks like).

Then remember what comes after: the policy gate replay (`backtest.py --gate`, stock over 60 days, at least 10 trades, Sharpe above zero, DSR at least 0.60), and, once a champion exists, 14 to 28 days of shadow comparison for any challenger. A certificate is a ticket to the next test, not a verdict of profitability.

14.7 A worked example: the arithmetic of skill

Grinold and Kahn's "fundamental law of active management" gives a rough rule: the information ratio (risk-adjusted excess return per year) is about the **information coefficient** (IC, the correlation between forecasts and outcomes) times the square root of **breadth** (the number of independent bets per year).

- An IC of 0.02, which is typical of a real but weak signal, with 2,500 independent bets a year gives $0.02 \times 50 = 1.0$ before costs. That would be excellent.
- The same IC with 100 independent bets a year gives $0.02 \times 10 = 0.2$. That is barely distinguishable from noise over several years.
- Now subtract costs. If each trade's expected gross edge is 0.3 % and the round trip costs 0.3 %, the net edge is zero regardless of breadth. If the true crypto cost is 0.3 % where 0.1 % was assumed, a model that looked profitable in the backtest is not.

This is why the diagnosis above keeps returning to two things: whether the model has any IC at all (the zero-skill comparison) and whether its trades clear their cost (the threshold versus the admission floor). Breadth cannot rescue a zero IC, and IC cannot rescue a trade that loses its edge to the spread.

14.8 The crypto decision

At 08:55 on 2026-09-27 the founder directed that effort stop on the crypto book and concentrate on stocks. The evidence behind it:

- 43 of 44 crypto trials negative, with an ordering explained by a random ranker;
- every crypto threshold below the live admission floor;
- realized crypto taker costs two to three times the modelled cost;
- six inherited crypto positions with a zero cost basis on a broker asset id that none of the system's orders had used, raising the question whether stop orders protected them at all.

The mechanics chosen are reversible: the service runs `run_pipeline.py --stock-only`, the paper account was liquidated to about \$122,000 in cash at 09:21, and nothing was deleted. Crypto-specific owner items (the spread census, funding and open-interest features, crypto de-risking) are on hold. If crypto is ever revisited, the first step is honest costs, not more trials.

14.9 What would change the verdict

Pre-registration means deciding in advance what evidence would change your mind. For this system:

- **Toward "there is skill":** a holdout certificate with DSR at or above 0.60 on a meaningful n_{eff} ; positive cross-sectional rank IC in the Stage-0 dump from the weekly backtest; a policy gate pass; and, over months, shadow and live measurements that agree.
- **Toward "stop":** the Phase-3 search, with the levers now on, again failing to beat a zero-skill ranker; or a certificate that passes but whose live results over the pre-registered horizon fall inside what luck predicts.
- **Not evidence either way:** in-search scores, a single good week of paper trading, or any verdict computed on journals written by an older policy.

14.10 Further reading

- Campbell R. Harvey, Yan Liu and Heqing Zhu, "... and the Cross-Section of Expected Returns," *Review of Financial Studies* 29(1), 2016, 5 to 68. Why the bar for a new finding should be a t-statistic near 3, not 2.
- Richard C. Grinold and Ronald N. Kahn, *Active Portfolio Management*, 2nd ed., McGraw-Hill, 2000. The fundamental law.
- Robert Novy-Marx and Mihail Velikov, "A Taxonomy of Anomalies and Their Trading Costs," *Review of Financial Studies* 29(1), 2016, 104 to 147. How costs erase high-turnover strategies.
- Ryan Sullivan, Allan Timmermann and Halbert White, "Data-Snooping, Technical Trading Rule Performance, and the Bootstrap," *Journal of Finance* 54(5), 1999, 1647 to 1691.
- In this repository: `research/campaign_2026-09_jetson/README.md` (the night's report), `research_signal.md`, the stock decomposition in `session_artifacts/decomp/RESULT_stock.md` [`research/campaign_2026-09_jetson/session_artifacts/decomp/RESULT_stock.md`], and the SIGNAL and ENGINE owner items under `session_artifacts/generals/` [`research/campaign_2026-09_jetson/session_artifacts/generals/signal/OWNER_ITEMS.md`].

Chapter 15. A reading path to financial literacy for this system

15.1 The idea

This book has explained one system. The ideas underneath it are general, and they are the ideas that separate people who understand markets from people who are merely exposed to them: what a return is made of, what trading costs, how to tell skill from luck, and how data lies. This last chapter is a map for going further. It has three parts: a crosswalk from the vocabulary of this book to the repository's glossary, a ten-item reading curriculum with a paragraph on why each item earns its place, and a field guide to the five traps that ruin most trading research, each shown with an example from this very repository.

15.2 Why it matters financially

Financial capability is not knowing which stock will rise. Nobody reliably knows that. It is the ability to ask the right questions of any claim, including your own: *Compared with what? After costs? On how much independent evidence? Using only information available at the time? Out of how many attempts?* A reader who asks those five questions by reflex will avoid most of the expensive mistakes in investing, whether the claim comes from a fund manager, a newsletter, an AI model or a backtest on a Jetson in a spare room.

15.3 How this system's vocabulary maps to the glossary

The repository's `docs/GLOSSARY.md` defines every term as it is used *here*, and names the module that owns it. The table points from the words used in Part Two to the glossary's alphabetical sections, where the entries live.

Term in this book	Glossary entry	Section	One-line meaning
book	book	B [docs/GLOSSARY.md#b]	one family of positions run by one loop (crypto or stock)
cycle, cooldown, lockout	cooldown; lockout	C [docs/GLOSSARY.md#c] , L [docs/GLOSSARY.md#l]	the 30 second loop and its per-symbol waiting rules
entry windows, EOD flatten	entry windows; EOD flatten	E [docs/GLOSSARY.md#e]	when stock entries are allowed; the close-out before 16:00 ET
overnight sleeve	overnight sleeve	O [docs/GLOSSARY.md#o]	up to two stock positions held overnight under strict rules
journal	journal	J [docs/GLOSSARY.md#j]	the append-only JSONL diary of every decision
fail-open, fail-closed	fail-open / fail-closed	F [docs/GLOSSARY.md#f]	what a component does when it breaks

Term in this book	Glossary entry	Section	One-line meaning
champion, challenger, shadow	champion; challenger; shadow slot	C [docs/GLOSSARY.md#c], S [docs/GLOSSARY.md#s]	the trading model, its silent rival, and where the rival lives
DM-HLN	DM-HLN	D [docs/GLOSSARY.md#d]	the forecast-comparison test used for promotion
DSR	DSR (Deflated Sharpe) / DSR_MIN	D [docs/GLOSSARY.md#d]	probability a Sharpe beats the best of N lucky trials
effective n	effective-n (n_eff)	E [docs/GLOSSARY.md#e]	independent observations hidden in correlated ones
purge, embargo	purge; embargo	P [docs/GLOSSARY.md#p], E [docs/GLOSSARY.md#e]	the gaps that stop training data leaking into validation
triple-barrier label	triple-barrier (TB) label	T [docs/GLOSSARY.md#t]	a label computed by walking the real exit rules
hot reload	hot-reload	H [docs/GLOSSARY.md#h]	bots swapping in a newly promoted model without restarting
gotcha #2	gotcha #2	G [docs/GLOSSARY.md#g]	reset the Optuna studies after any objective or feature change
default-OFF, evidence gate	default-OFF; evidence gate	D [docs/GLOSSARY.md#d], E [docs/GLOSSARY.md#e]	how model-facing changes are shipped and switched on
beta ledger, AKL lagged beta	beta ledger; AKL lagged beta	B [docs/GLOSSARY.md#b], A [docs/GLOSSARY.md#a]	measuring market exposure, including delayed exposure
b2, echo gap	b2; echo gap	B [docs/GLOSSARY.md#b], E [docs/GLOSSARY.md#e]	whether the LLM adds anything beyond the model
LLM veto, veto strike, llm_mult	LLM veto; veto strike; llm_mult	L [docs/GLOSSARY.md#l], V [docs/GLOSSARY.md#v]	the LLM's bounded powers
selection mode, free-first	selection mode; free-first	S [docs/GLOSSARY.md#s], F [docs/GLOSSARY.md#f]	how the LLM provider is chosen
round-trip cost, required edge	round-trip cost; required edge	R [docs/GLOSSARY.md#r]	what a trade must earn to break even
survivorship, PIT	survivorship; PIT (point-in-time)	S [docs/GLOSSARY.md#s], P [docs/GLOSSARY.md#p]	two of the traps in section 15.5
MinTRL	MinTRL	M [docs/GLOSSARY.md#m]	how long a track record must be to prove a Sharpe ratio

15.4 A ten-item curriculum

The order runs from foundations to the specific methods this system uses. None requires advanced mathematics beyond what the chapters of this book have already shown; several are written for practitioners. Read them in order if you can; each one makes the next easier.

1. **Zvi Bodie, Alex Kane and Alan J. Marcus, *Investments* (McGraw-Hill, any recent edition).** The standard university textbook. Read the chapters on risk and return, the capital asset pricing model, index models and market efficiency. It gives you the language of this book (return, volatility, beta, alpha, diversification) with worked problems, and it teaches the discipline of asking what a return is *compensation for*. Everything else on this list assumes it.
2. **Larry Harris, *Trading and Exchanges: Market Microstructure for Practitioners* (Oxford University Press, 2003).** How orders actually become trades: limit and market orders, spreads, dealers, informed and uninformed traders, and why the price you see is not the price you get. After this book the cost gate, the maker ladder, stale quotes and the crypto spread finding in Chapter 14 will make intuitive sense.
3. **Antti Ilmanen, *Expected Returns: An Investor's Guide to Harvesting Market Rewards* (Wiley, 2011).** A survey of where long-run returns come from across asset classes and strategies: equity premium, value, carry, momentum, volatility selling, illiquidity. It teaches humility about forecasting and respect for simple, well-documented premia, which is the right frame for judging whether an hourly LSTM should be expected to add anything.
4. **Richard C. Grinold and Ronald N. Kahn, *Active Portfolio Management*, 2nd ed. (McGraw-Hill, 2000).** The quantitative manager's handbook. Its fundamental law (Chapter 14's worked example) explains why small forecasting skill applied widely beats large skill applied rarely, and why costs and breadth dominate the arithmetic. Dense in places; the early chapters and the chapter on the fundamental law are the essentials.
5. **Andrew W. Lo, "The Statistics of Sharpe Ratios," *Financial Analysts Journal* 58(4), 2002.** A short, readable paper that shows the Sharpe ratio is an *estimate* with a standard error, that annualizing it by a square-root rule is often wrong, and that serial correlation changes everything. It is the natural bridge from Chapter 11 to the more technical papers below.
6. **David Aronson, *Evidence-Based Technical Analysis* (Wiley, 2006).** Written for traders, not academics, it explains the scientific method applied to trading rules and devotes a chapter to data-mining bias, "the fool's gold of objective technical analysis". It is the gentlest complete introduction to why testing many rules and keeping the best one guarantees a disappointing future.
7. **David H. Bailey and Marcos López de Prado, "The Deflated Sharpe Ratio," *Journal of Portfolio Management* 40(5), 2014.** The source of this system's promotion gate. Once you have read Lo and Aronson, this paper's single idea (compare your Sharpe with the best that luck would produce from as many trials as you ran) will feel inevitable. Pair it with the worked table in Chapter 10.
8. **Marcos López de Prado, *Advances in Financial Machine Learning* (Wiley, 2018).** The practitioner's reference for most of this repository's methods: triple-barrier labels, meta-labeling, sample uniqueness and effective sample size, purged and embargoed cross-validation, backtest overfitting. Read it as a manual of hazards rather than a recipe book; its value is in the failure modes it names.
9. **Campbell R. Harvey, Yan Liu and Heqing Zhu, "... and the Cross-Section of Expected Returns," *Review of Financial Studies* 29(1), 2016.** Hundreds of published return predictors, examined as a multiple-testing problem. The conclusion (a new factor should clear a t-statistic of about 3, not 2) is the academic version of this system's DSR gate. Read with McLean and Pontiff (2016), who measured how much published anomalies shrink after publication.

10. Ernest P. Chan, *Quantitative Trading: How to Build Your Own Algorithmic Trading Business*, 2nd ed. (Wiley, 2021). A small operator's view: data, backtesting pitfalls, execution, risk, capacity and the practical business of running a strategy. It is the closest book on this list to the founder's actual situation, a one-person system with real infrastructure constraints, and it is candid about how often strategies fail.

A bonus for the habit of mind rather than the mathematics: Daniel Kahneman, *Thinking, Fast and Slow* (Farrar, Straus and Giroux, 2011), on why people see patterns in noise and trust small samples. Every trap below is a cognitive bias before it is a statistical one.

15.5 The five traps, with examples from this repository

Trap 1: Overfitting

What it is. Fitting the noise in past data so well that the model fails on new data. Its common form in trading is selection: try many configurations, keep the best.

Where it appeared here. The overnight crypto search ran 44 configurations. One scored slightly positive. Without the DSR gate that one would have looked like "the model that works".

The defence. Walk-forward folds with purging and embargo, a holdout never seen by the search, the DSR gate with the true number of trials, and shadow comparison on live data.

The question to ask. "Out of how many attempts?"

Trap 2: Look-ahead bias

What it is. Using information in a test that was not available at the moment of the decision.

Where it appeared here. The `Daily_Sentiment` feature was keyed by host-local date, so every row held the next day's Fear and Greed value. Every training store built before 2026-09-26 carried a one-day look-ahead, and the April models were trained on it. It was found by an audit and repaired in the clean rebuild. LLMs add a subtler version: a model scoring a 2023 headline may already know what happened next.

The defence. Strictly trailing features computed by the same functions at harvest and live, publication-date lags for sentiment and short interest, and judging LLM opinions only on live, post-cutoff decisions.

The question to ask. "Could I have known this at the time?"

Trap 3: Survivorship bias

What it is. Testing on the names that exist today, which silently excludes the ones that failed. Brown, Goetzmann, Ibbotson and Ross (1992) showed it can even create the appearance of predictability where there is none.

Where it is prevented here. The stock harvest keeps a name on a given date only if it was among the top 60 by 30-day dollar volume *on that date* (`_asof_membership_mask`), not today. The repository's invariants call this mask the only place survivorship is prevented and forbid "simplifying" it.

A worked example. Take 100 stocks. Over a year, 30 fall 50 % and 70 rise 21.4 %. The average stock returned $0.7 \times 21.4 \% + 0.3 \times (-50 \%) = 0 \%$. Now suppose the fallers dropped out of the top-volume list and you build your test universe from today's list. Your universe holds mostly risers, and "buy everything" shows about +21 %, from a strategy with no skill at all.

The question to ask. "Which names were in the universe *then*?"

Trap 4: Cost blindness

What it is. Ignoring or understating the cost of trading, which punishes high-turnover strategies most (Novy-Marx and Velikov 2016).

Where it appeared here. The crypto cost model assumes a 10 basis point spread; reconstructed paper fills showed 26 to 34 basis points round trip on three of the six coins (Chapter 14). Separately, the crypto search kept choosing thresholds below the live admission floor, so its simulated trades were ones the live cost gate would never have taken.

The defence. One cost function (`fees.round_trip_cost_pct`) shared by the backtester, the meta-labeler and the live gate; realised-cost reports (`execution_report.py`, `scripts/fill_venue_slippage_report.py`); and the rule that a new gate re-deriving cost is a bug.

The question to ask. "After all costs, including the spread I actually paid?"

Trap 5: Small samples

What it is. Drawing conclusions from too little independent evidence, often disguised by many correlated rows.

Where it appeared here. The stock holdout had 20 trades, 14 effectively independent (Chapter 10). The LLM scorecard's original test rejected a true null two to three times too often at 20 to 60 effective observations (Chapter 11). An alpha estimate needed about two years of data against a window under half a year.

The defence. Effective-n everywhere, pre-registered minimum counts before any verdict, "cannot conclude" as an honest output, and anytime-valid methods when you must look repeatedly.

The question to ask. "How many *independent* observations is this?"

15.6 What the evidence says about learning this way

There is no controlled trial showing that reading these ten items makes anyone a better investor. What the evidence does show is how often the five traps above explain published and professional failures: McLean and Pontiff's post-publication decay, Harvey, Liu and Zhu's multiple-testing arithmetic, Sullivan, Timmermann and White's finding that data-snooping explains much of the apparent success of technical rules, and this repository's own overnight result. Learning to recognise those traps is the most reliable financial skill there is, because it protects capital regardless of what the market does next.

15.7 Further reading

The curriculum above is the further reading. Two companions for the traps:

- Stephen J. Brown, William Goetzmann, Roger G. Ibbotson and Stephen A. Ross, "Survivorship Bias in Performance Studies," *Review of Financial Studies* 5(4), 1992, 553 to 580.
- R. David McLean and Jeffrey Pontiff, "Does Academic Research Destroy Stock Return Predictability?" *Journal of Finance* 71(1), 2016, 5 to 32.

And inside the repository, in reading order for a human: `docs/README.md`, then `docs/MAP.md` sections 1 and 2, then `docs/GLOSSARY.md` as needed, and `research/KILL_LIST.md` for the ideas that were tried and retired, with the reasons.